

Exact inference on trees

The calculation your speech model runs on every training step

Preparation edition

DSAA 4011 | Fall 2026–27

Yuwen Huang | Week 2 lecture

What we will compute today

A model is a product of small tables.

Three machines, each alerting (1) or quiet (0), and a table of weights per coupled pair, as in Week 1.

What we will compute today

A model is a product of small tables.

We want every **group total**: for each variable, for each of its values,
the total weight of every case that has it.

A case, an **assignment**: one value for every variable at once, such as $(x_1, x_2, x_3) = (1, 0, 1)$. Week 1 wrote it $g(x_Q)$ and listed every case.

What we will compute today

A model is a product of small tables.

We want every **group total**: for each variable, for each of its values,
the total weight of every case that has it.

Exactly. And with work that grows with the number of variables, not with the number of cases.

Three machines have 8 cases; 40 would have 10^{12}. That is the whole difficulty.

What we will prove today

When the drawing of the model is a tree,
two sweeps of short lists along its edges
deliver, at every variable, exactly its group totals.

What we will prove today

When the drawing of the model is a tree,
two sweeps of short lists along its edges
deliver, at every variable, exactly its group totals.

Every slide today is either a step toward this sentence or a consequence of it. Part 3 proves it; Parts 1 and 2 build the lists it speaks of, by hand, on one running model.

The map: six steps, one argument

1. Why: a loss that is a sum with 10^{338} terms
2. One step by hand: pull the shared factor out, close a box
3. The whole method on a tree: every box becomes a list on an edge; two sweeps
4. Why it is exact: the theorem, its conditions, how it breaks
5. The same theorem with $\max$; with a cycle; with continuous values
6. Where it is used

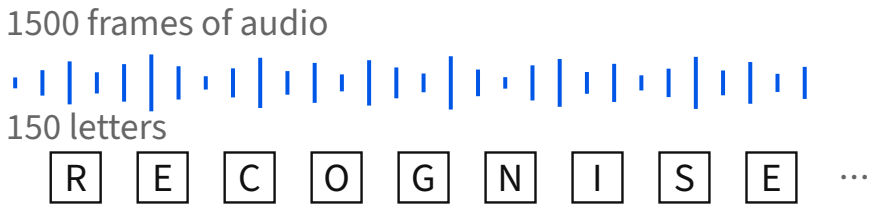
This map returns at every part boundary. Step 1: why anyone needs such a theorem.

Assumptions: what Week 1 gave, and what is not needed today

From Week 1. Finite value sets; tables of weights ≥ 0; an assignment's weight is their product; totals by listing every assignment; $C = 30$ with Machine 3 known.
Not needed. Probability beyond one ratio; induction; code.

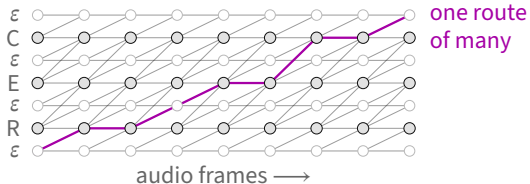
last week	this week	
$w(x)$	$g(x)$	the same product
$g(x_Q)$	$g_i(x_i)$	**the letter moved**: the group total
factor	local function	the same square in the drawing
x_1	X_1 and x_1	the quantity, and one of its values
C	Z	every weight, added

Thirty seconds of speech, one transcript, and no alignment



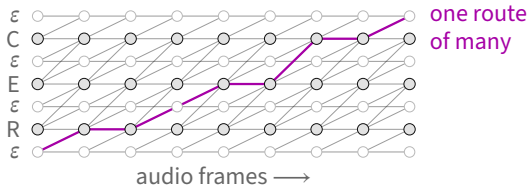
Nobody told the model which frame produced which letter. How does it score the transcript at all?

The lattice, one part at a time



Frame: one column, 20 ms of audio.

The lattice, one part at a time

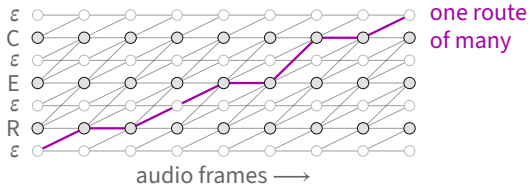


Frame: one column, 20 ms of audio.

Symbol: one row; a letter, or a blank ε.

Blank: silence, or the gap between letters; without it R R reads as one R.

The lattice, one part at a time

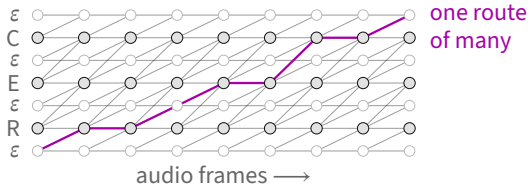


Frame: one column, 20 ms of audio.

Symbol: one row; a letter, or a blank ε.

Cell: one circle, a frame and a symbol.

The lattice, one part at a time



Frame: one column, 20 ms of audio.

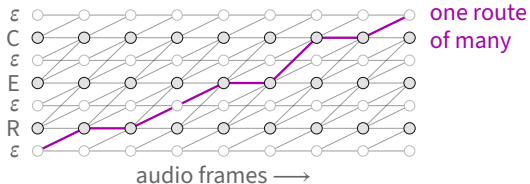
Route: left to right, one cell per frame.

Symbol: one row; a letter, or a blank ε.

Cell: one circle, a frame and a symbol.

A route is one alignment: frame by frame, which symbol is said. The magenta one is one of many.

The lattice, one part at a time



Frame: one column, 20 ms of audio.

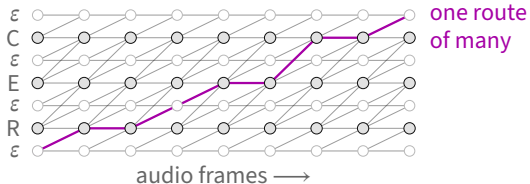
Route: left to right, one cell per frame.

Symbol: one row; a letter, or a blank ε.

Move: stay on the row, or advance one row.

Cell: one circle, a frame and a symbol.

The lattice, one part at a time



Frame: one column, 20 ms of audio.

Route: left to right, one cell per frame.

Symbol: one row; a letter, or a blank ε.

Move: stay on the row, or advance one row.

Cell: one circle, a frame and a symbol.

Skip: over a blank, if the letters differ.

Three moves, every frame. Which route does the model pick?

The network picks no route: it scores every symbol at every frame

Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3$, $y_1(\mathrm{R}) = 0.7$; $y_2(\varepsilon) = 0.4$, $y_2(\mathrm{R}) = 0.6$.



One score per symbol per frame; no letter is ever assigned to a frame.

The network picks no route: it scores every symbol at every frame

Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3$, $y_1(\mathrm{R}) = 0.7$; $y_2(\varepsilon) = 0.4$, $y_2(\mathrm{R}) = 0.6$.



RR : $0.7 \cdot 0.6$

One route: R at both frames. Frames are scored independently, so multiply.

The network picks no route: it scores every symbol at every frame

Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3$, $y_1(\mathrm{R}) = 0.7$; $y_2(\varepsilon) = 0.4$, $y_2(\mathrm{R}) = 0.6$.

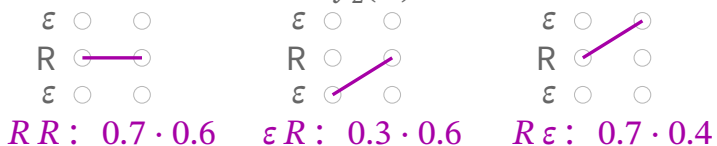


$R\,R:\ 0.7 \cdot 0.6$ $\varepsilon\,R:\ 0.3 \cdot 0.6$

A second route spells the same “R”.

The network picks no route: it scores every symbol at every frame

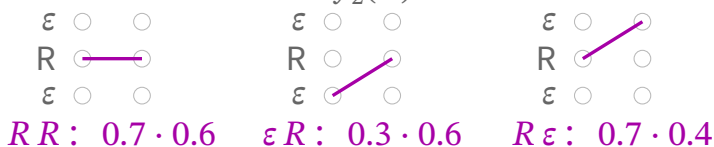
Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3$, $y_1(\mathrm{R}) = 0.7$; $y_2(\varepsilon) = 0.4$, $y_2(\mathrm{R}) = 0.6$.



No fourth: start on ε or R, end on R or the last ε, pass through R.

The network picks no route: it scores every symbol at every frame

Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3, y_1(\mathrm{R}) = 0.7; y_2(\varepsilon) = 0.4, y_2(\mathrm{R}) = 0.6.$

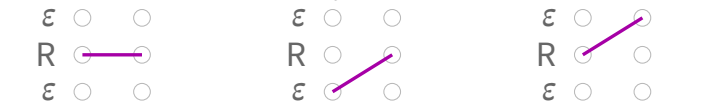


$$p(\text{“R”}) = 0.7 \cdot 0.6 + 0.3 \cdot 0.6 + 0.7 \cdot 0.4 = 0.88$$

Different routes cannot both happen, so add. p(“R”): the probability the model gives to the transcript.

The network picks no route: it scores every symbol at every frame

Transcript “R”, two frames. Scores: $y_1(\varepsilon) = 0.3$, $y_1(\mathrm{R}) = 0.7$; $y_2(\varepsilon) = 0.4$, $y_2(\mathrm{R}) = 0.6$.



$R\,R$: $0.7 \cdot 0.6$ $\quad$ $\varepsilon\,R$: $0.3 \cdot 0.6$ $\quad$ $R\,\varepsilon$: $0.7 \cdot 0.4$

$$p(\text{“R”}) = 0.7 \cdot 0.6 + 0.3 \cdot 0.6 + 0.7 \cdot 0.4 = 0.88$$

Training raises 0.88; the loss is $-\log 0.88$. How many routes does the real lattice hold?

How many routes does the small lattice hold, and the full one?

Each letter needs one advance into it and one out of it.
Three letters: 6 advances, spread over 8 frames.

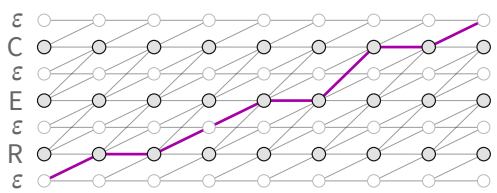
Predict before calculating.

How many routes does the small lattice hold, and the full one?

Each letter needs one advance into it and one out of it.
Three letters: 6 advances, spread over 8 frames.

Predict before calculating.

The same sum at full size: multiply along a route, add over routes

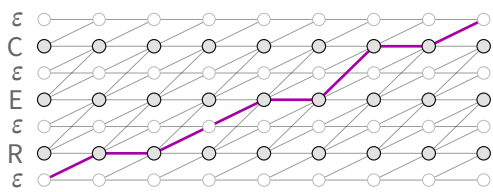


$$\text{one route} = \prod_{t=1}^{1500} y_t(s_t)$$

1500 scores, multiplied

s_t is the symbol the route sits on at frame t. The magenta route multiplies nine scores; a real one, 1500.

The same sum at full size: multiply along a route, add over routes



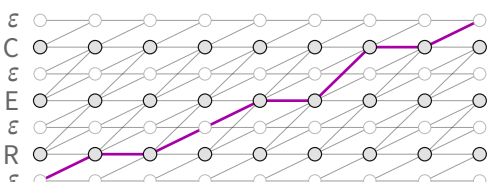
one route of many

$$\text{one route} = \prod_{t=1}^{1500} y_t(s_t)$$ 1500 scores, multiplied

$$p(\text{transcript}) = \sum_{\text{routes}} \prod_{t=1}^{1500} y_t(s_t)$$ 10^{338} terms, added

p(transcript): the probability the model gives to the whole 150-letter transcript, the p("R") of the last slide at full size.

The same sum at full size: multiply along a route, add over routes



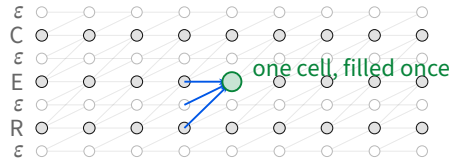
one route of many

$$\text{one route} = \prod_{t=1}^{1500} y_t(s_t) \qquad \text{1500 scores, multiplied}$$

$$p(\text{transcript}) = \sum_{\text{routes}} \prod_{t=1}^{1500} y_t(s_t) \qquad 10^{338} \text{ terms, added}$$

Nobody can add 10^{338} products one by one. So how is it done?

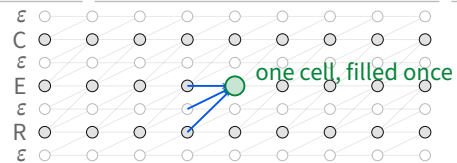
A cell stores a running total, so no route is ever listed



$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$

Call $\alpha_t(s)$ the total weight of every partial route that reaches the cell (t, s).

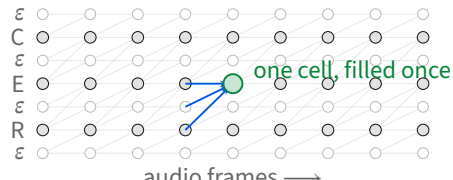
A cell stores a running total, so no route is ever listed



$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$

Every one of those routes ends on the same cell, so every one of them ends with the same factor $y_t(s)$.

A cell stores a running total, so no route is ever listed

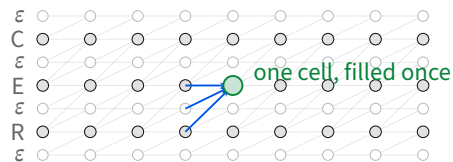


$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$

$$= \Big(\sum_{\text{routes to } (t,s)} \prod_{u<t} y_u(s_u)\Big)\, y_t(s)$$

Pull that shared factor out of the sum. This is the move of the whole lecture.

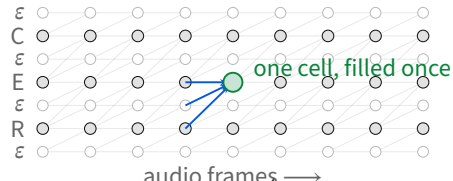
A cell stores a running total, so no route is ever listed



$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$
$$= \Big(\sum_{\text{routes to } (t,s)} \prod_{u<t} y_u(s_u)\Big)\, y_t(s)$$

The step before (t, s) was a stay, an advance or a skip: the route was on one of three cells.

A cell stores a running total, so no route is ever listed



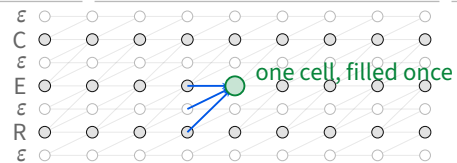
$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$

$$= \Big(\sum_{\text{routes to } (t,s)} \prod_{u<t} y_u(s_u)\Big)\, y_t(s)$$

$$= \big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2)\big)\, y_t(s)$$

So the part in parentheses is three earlier cells, added.

A cell stores a running total, so no route is ever listed



$$\alpha_t(s) = \sum_{\text{routes to } (t,s)} \prod_{u=1}^{t} y_u(s_u)$$
$$= \Big(\sum_{\text{routes to } (t,s)} \prod_{u<t} y_u(s_u)\Big)\, y_t(s)$$
$$= \big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2)\big)\, y_t(s)$$

$1500 \times 301 = 451\,500$ cells, filled left to right; the last one is p(transcript).
Why is that exact?

That sum is the CTC loss; the rest of today explains why the cells get it right

Where	`torch.nn.CTCLoss`, a cuDNN kernel. It fills the $451\,500$ cells.

That sum is the CTC loss; the rest of today explains why the cells get it right

Where	`torch.nn.CTCLoss`, a cuDNN kernel. It fills the $451\,500$ cells.
Returns	$-\log p(\text{transcript})$ and its gradient.

That sum is the CTC loss; the rest of today explains why the cells get it right

Where	`torch.nn.CTCLoss`, a cuDNN kernel. It fills the $451\,500$ cells.
Returns	$-\log p(\text{transcript})$ and its gradient.
Question	Why do the cells give exactly the sum over all 10^{338} routes? And on what kind of model does the same trick fail?

That sum is the CTC loss; the rest of today explains why the cells get it right

Where	`torch.nn.CTCLoss`, a cuDNN kernel. It fills the $451\,500$ cells.
Returns	$-\log p(\text{transcript})$ and its gradient.
Question	Why do the cells give exactly the sum over all 10^{338} routes? And on what kind of model does the same trick fail?
Method	Steps 2 to 4 of the map, on a model small enough to check by hand: three machines, two tables, eight cases. The trick is the same: pull the shared factor out.

1. One step by hand: pull the shared factor out

On three machines: is there a shortcut to the group totals, and why is it allowed?

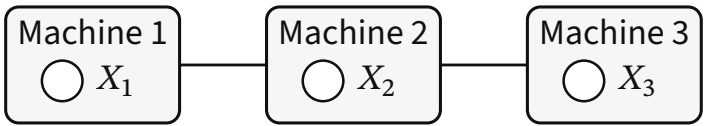
You will do it by hand, and check that the answer did not move.

Where we are

1. Why: a loss that is a sum with 10^{338} terms
2. **One step by hand: pull the shared factor out, close a box**
3. The whole method on a tree: every box becomes a list on an edge; two sweeps
4. Why it is exact: the theorem, its conditions, how it breaks
5. The same theorem with $\max$; with a cycle; with continuous values
6. Where it is used

In hand: the target, and one move seen on the lattice. This part does that move once, slowly, where every number can be checked.

Three machines share one supply line

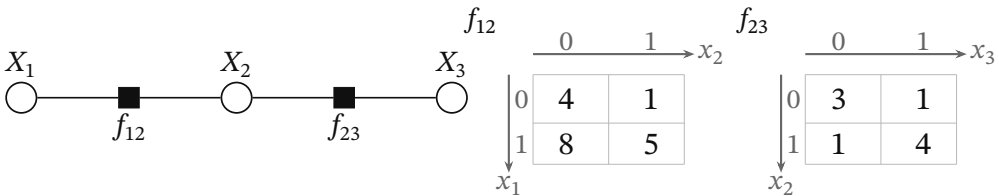


each $X_i \in \{0, 1\}$: 0 is quiet, 1 is alerting

neighbours share a supply line, so their states are not independent

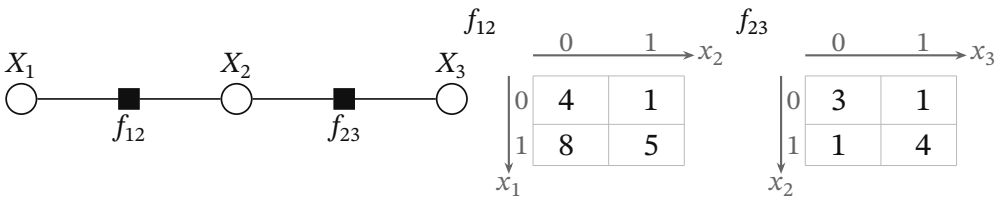
Machines 1 and 2 share power, 2 and 3 share coolant: not independent, so the coupling needs numbers. What do the numbers say?

Two tables, and one number for each complete assignment



The first argument is the row, the second the column. That is the convention.

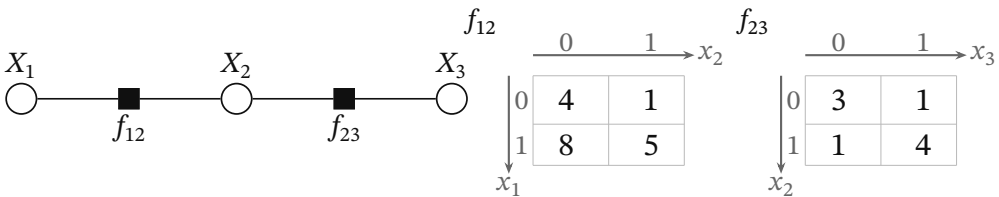
Two tables, and one number for each complete assignment



$$g(x_1, x_2, x_3) = f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)$$

The weight of one complete assignment: what each table says about its pair, multiplied.

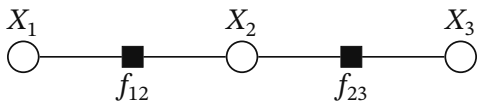
Two tables, and one number for each complete assignment



$$g(x_1, x_2, x_3) = f_{12}(x_1, x_2)\, f_{23}(x_2, x_3), \quad \text{so} \quad g(1, 0, 1) = 8 \cdot 1 = 8$$

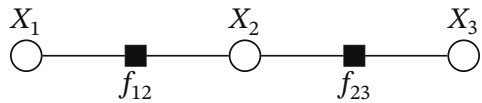
$f_{12}(1, 0) = 8$ was 2 last week; you will see why in ten minutes. First, the object in one sentence.

The picture and the product say the same thing



Definition 1 (factorised model). Each variable X_i has a finite set of values $\mathcal{X}_i$, here $\{0, 1\}$.

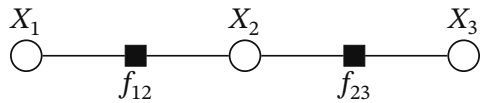
The picture and the product say the same thing



Definition 1 (factorised model). Each variable X_i has a finite set of values $\mathcal{X}_i$, here $\{0, 1\}$. A **local function** f reads the variables listed in $\mathcal{N}(f)$ and returns a weight ≥ 0.

Example: f_{12} reads X_1 and X_2, so $\mathcal{N}(f_{12}) = \{1, 2\}$ and it has two edges; $f_{12}(1, 0) = 8$ is one weight.

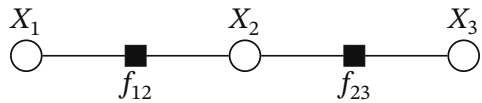
The picture and the product say the same thing



Definition 1 (factorised model). Each variable X_i has a finite set of values $\mathcal{X}_i$, here $\{0, 1\}$. A **local function** f reads the variables listed in $\mathcal{N}(f)$ and returns a weight ≥ 0. An edge joins f to each variable it reads.

Example: f_{12} reads X_1 and X_2, so $\mathcal{N}(f_{12}) = \{1, 2\}$ and it has two edges; $f_{12}(1, 0) = 8$ is one weight.

The picture and the product say the same thing



Definition 1 (factorised model). Each variable X_i has a finite set of values $\mathcal{X}_i$, here $\{0, 1\}$. A **local function** f reads the variables listed in $\mathcal{N}(f)$ and returns a weight ≥ 0. An edge joins f to each variable it reads. The **global function** $g(x) = \prod_f f(x_{\mathcal{N}(f)})$ is the weight of one complete assignment.

Example: $g(1, 0, 1) = 8 \cdot 1 = 8$. A weight is not a probability: f_{12}'s entries add to 18, like a network's output scores. Now the target, on this object.

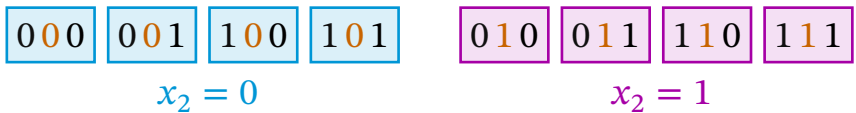
The target on this model: the total weight of each group, sorted by x_2

all eight assignments (x_1, x_2, x_3)

000	001	010	011	100	101	110	111

Week 1's way: list all eight assignments.

The target on this model: the total weight of each group, sorted by x_2



each group is a separate rectangle picture

$$g_2(x_2) = \sum_{x_1} \sum_{x_3} f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)$$

Sort them by x_2 and add each group: two totals. Is there a shortcut?

The group for $x_2 = 0$, added up in five lines

$$g_2(0) = \sum_{x_1} \sum_{x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) \quad \text{fix } x_2 = 0$$

The group for $x_2 = 0$, added up in five lines

$$g_2(0) = \sum_{x_1} \sum_{x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) \quad \text{fix } x_2 = 0$$

$$= 4 \cdot 3 + 4 \cdot 1 + 8 \cdot 3 + 8 \cdot 1 \quad \text{one term per } (x_1, x_3)$$

Line 2 costs seven operations: four products, three sums.

The group for $x_2 = 0$, added up in five lines

$$
\begin{aligned}
g_2(0) &= \textstyle\sum_{x_1} \sum_{x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) && \text{fix } x_2 = 0 \\
&= 4 \cdot 3 + 4 \cdot 1 + 8 \cdot 3 + 8 \cdot 1 && \text{one term per } (x_1, x_3) \\
&= 4\,(3 + 1) + 8\,(3 + 1) && \text{pull the first factor out}
\end{aligned}
$$

The group for $x_2 = 0$, added up in five lines

$$
\begin{aligned}
g_2(0) &= \textstyle\sum_{x_1} \sum_{x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) && \text{fix } x_2 = 0 \\
&= 4 \cdot 3 + 4 \cdot 1 + 8 \cdot 3 + 8 \cdot 1 && \text{one term per } (x_1, x_3) \\
&= 4\,(3 + 1) + 8\,(3 + 1) && \text{pull the first factor out} \\
&= \underbrace{(4 + 8)}_{\ell(0)}\ \underbrace{(3 + 1)}_{r(0)} && \text{both pairs share } (3 + 1)
\end{aligned}
$$

The two named pieces are called **brackets**.

The group for $x_2 = 0$, added up in five lines

$$g_2(0) = \sum_{x_1} \sum_{x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) \qquad \text{fix } x_2 = 0$$

$$= 4 \cdot 3 + 4 \cdot 1 + 8 \cdot 3 + 8 \cdot 1 \qquad \text{one term per } (x_1, x_3)$$

$$= 4\,(3 + 1) + 8\,(3 + 1) \qquad \text{pull the first factor out}$$

$$= \underbrace{(4 + 8)}_{\ell(0)}\ \underbrace{(3 + 1)}_{r(0)} \qquad \text{both pairs share } (3 + 1)$$

$$= 12 \cdot 4 = 48 \qquad \text{two sums, one product}$$

Line 5 costs three operations. Same number. Is line 4 always allowed?

Line 4 is an identity, and it is the only algebra today needs

Lemma 1 (distributive law for finite sums). $\sum_{s}\sum_{t} a_s b_t = \Big(\sum_{s} a_s\Big)\Big(\sum_{t} b_t\Big).$

Lines 3 and 4 used it, with $a = (4, 8)$ and $b = (3, 1)$. Nothing is approximated: both sides are the same number.

Line 4 is an identity, and it is the only algebra today needs

Lemma 1 (distributive law for finite sums). $\sum_s \sum_t a_s b_t = \Big(\sum_s a_s\Big)\Big(\sum_t b_t\Big).$

The same identity holds with $\max$ in place of $+$, step 5 of the map. Does it work for $x_2 = 1$ too?

The group for $x_2 = 1$, added up by the same three moves

$$g_2(1) = 1 \cdot 1 + 1 \cdot 4 + 5 \cdot 1 + 5 \cdot 4$$ one term per (x_1, x_3)

The group for $x_2 = 1$, added up by the same three moves

$$g_2(1) = 1 \cdot 1 + 1 \cdot 4 + 5 \cdot 1 + 5 \cdot 4 \quad \text{one term per } (x_1, x_3)$$

$$= \underbrace{(1+5)}_{\ell(1)} \underbrace{(1+4)}_{r(1)} \quad \text{Lemma 1, twice}$$

Each bracket is a column or a row of one table, added up.

The group for $x_2 = 1$, added up by the same three moves

$$g_2(1) = 1 \cdot 1 + 1 \cdot 4 + 5 \cdot 1 + 5 \cdot 4 \qquad \text{one term per } (x_1, x_3)$$

$$= \underbrace{(1+5)}_{\ell(1)} \underbrace{(1+4)}_{r(1)} \qquad \text{Lemma 1, twice}$$

$$= 6 \cdot 5 = 30 \qquad \text{two sums, one product}$$

The two group totals are $(48, 30)$. What did we compute along the way?

Each bracket is a short list: one number per value of x_2

$$g_2(x_2) = \underbrace{\left(\sum_{x_1} f_{12}(x_1, x_2)\right)}_{\ell(x_2)} \underbrace{\left(\sum_{x_3} f_{23}(x_2, x_3)\right)}_{r(x_2)}$$

Line 4 of both chains, written for either value of x_2 at once.

Each bracket is a short list: one number per value of x_2

	$\ell(x_2) = \sum_{x_1} f_{12}(x_1, x_2)$	$r(x_2) = \sum_{x_3} f_{23}(x_2, x_3)$
$x_2 = 0$	$4 + 8 = 12$	$3 + 1 = 4$
$x_2 = 1$	$1 + 5 = 6$	$1 + 4 = 5$

The same sum, done once for each value of x_2.

Each bracket is a short list: one number per value of x_2

$\ell(x_2) = \sum_{x_1} f_{12}(x_1, x_2)$ $\quad$ $r(x_2) = \sum_{x_3} f_{23}(x_2, x_3)$

$x_2 = 0$ $\quad 4 + 8 = 12$ $\quad$ $3 + 1 = 4$

$x_2 = 1$ $\quad 1 + 5 = 6$ $\quad$ $1 + 4 = 5$

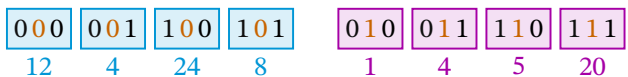
$\ell =$ | 12 | 6 | $\quad$ $r =$ | 4 | 5 |

one entry per value of x_2, because the sum was done once for each

$\ell = (12, 6)$, $r = (4, 5)$: two entries each, because $|\mathcal{X}_2| = 2$. Their totals, 18 and 9, mean nothing. From totals to shares?

Weights become shares only after dividing by their total

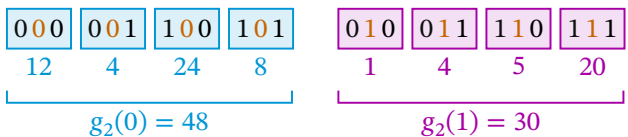
$g(x) = f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)$, one weight per assignment



Eight assignments, eight weights, from the two tables.

Weights become shares only after dividing by their total

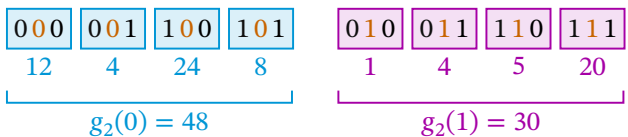
$g(x) = f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)$, one weight per assignment



The two group totals, as on the chains.

Weights become shares only after dividing by their total

$g(x) = f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)$, one weight per assignment



$$Z = 48 + 30 = 78$$

$$p(x_2) = \left(\frac{48}{78}, \frac{30}{78}\right) = \left(\frac{8}{13}, \frac{5}{13}\right)$$

Divide each group total by the total of everything.

Weights become shares only after dividing by their total

$Z = 48 + 30 = 78$ both groups, added: every weight once

Weights become shares only after dividing by their total

$$Z = 48 + 30 = 78$$ both groups, added: every weight once

$$p(x_2{=}0) = 48/78 \approx 0.62$$ the first group's share

Weights become shares only after dividing by their total

$$Z = 48 + 30 = 78$$ both groups, added: every weight once

$$p(x_2{=}0) = 48/78 \approx 0.62$$ the first group's share

$$p(x_2{=}1) = 30/78 \approx 0.38$$ the second group's share

That division is the softmax you have written many times, over two groups instead of fifty thousand classes.

The total and the group total both have names

Definition 2 (partition function and marginal). $Z = \sum_x g(x)$ is the **partition function**: every weight, added. $g_i(x_i)$ is the group total for $X_i = x_i$: every other variable summed away. The **marginal** is the share $p(x_i) = g_i(x_i)/Z$.

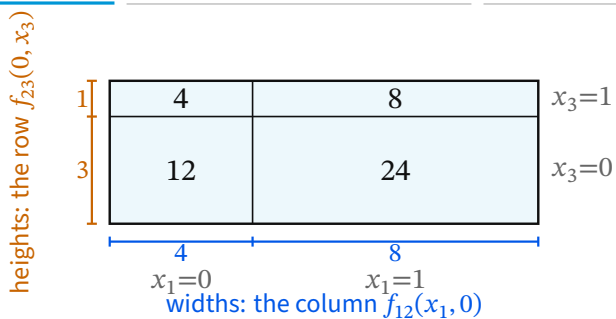
Both were computed on the previous slide, before either had a name: $Z = 78$, $g_2 = (48, 30)$, $p = (0.62, 0.38)$. The target is reached on three machines.

The total and the group total both have names

Definition 2 (partition function and marginal). $Z = \sum_x g(x)$ is the **partition function**: every weight, added. $g_i(x_i)$ is the group total for $X_i = x_i$: every other variable summed away. The **marginal** is the share $p(x_i) = g_i(x_i)/Z$.

Only the final answer is divided by Z; a bracket never is. Was the shortcut worth it?

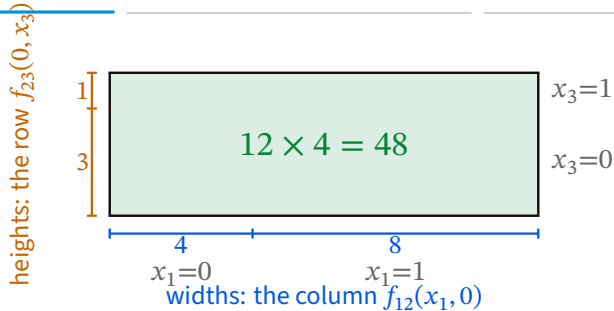
Lines 2 and 5 of the chain, as a picture



$$4 \cdot 3 + 4 \cdot 1 + 8 \cdot 3 + 8 \cdot 1 = 48$$

Four rectangles, added: seven operations.

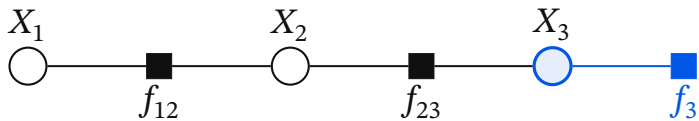
Lines 2 and 5 of the chain, as a picture



$$(4 + 8)(3 + 1) = 12 \cdot 4 = 48$$

One rectangle: three operations, and the move the lattice makes 451 500 times. Now change the model: Machine 3 alerts.

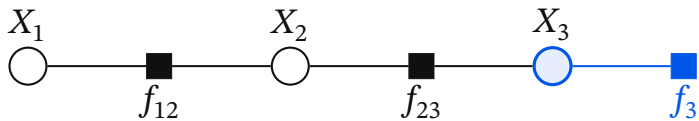
Now Machine 3 raises an alert



Observing $X_3 = 1$ multiplies in one more table, $f_3 = (0, 1)$.

Which of the two brackets has to be recomputed?

Now Machine 3 raises an alert



Observing $X_3 = 1$ multiplies in one more table, $f_3 = (0, 1)$.

Which of the two brackets has to be recomputed?

Now suppose Machine 3 had reported quiet instead

ℓ | 12 | 6

Observing $X_3 = 0$: the table is now $f_3 = (1, 0)$, one at $x_3 = 0$, zero at $x_3 = 1$.

How much of the work has to be done again?

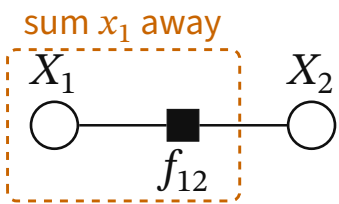
Now suppose Machine 3 had reported quiet instead

ℓ | 12 | 6

Observing $X_3 = 0$: the table is now $f_3 = (1, 0)$, one at $x_3 = 0$, zero at $x_3 = 1$.

How much of the work has to be done again?

The left branch is now two numbers on one edge

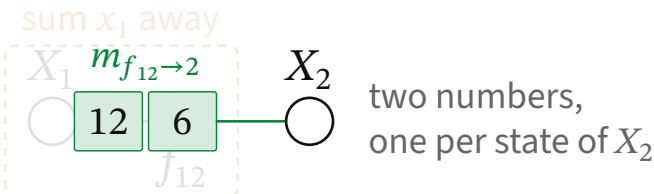


two nodes and
a four-entry table

a branch: everything hanging off X_2 on one side

The branch holds two nodes and a four-entry table.

The left branch is now two numbers on one edge



$\ell = (12, 6)$, a function of x_2 alone

We will call this **closing the box**: a whole branch replaced by one short list.

Does it repeat?

Closing the box again and again: a chain of four machines

$$g_4(x_4) = \sum_{x_1,x_2,x_3} f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)\, f_{34}(x_3, x_4)$$

One more machine, one more table. Listing the cases: 16 assignments, 8 per value of x_4.

Closing the box again and again: a chain of four machines

$$g_4(x_4) = \sum_{x_1,x_2,x_3} f_{12}(x_1,x_2)\, f_{23}(x_2,x_3)\, f_{34}(x_3,x_4)$$
$$= \sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3) \sum_{x_1} f_{12}(x_1,x_2)$$

Lemma 1 twice: x_1 sits in one table only, so its sum moves up to that table; the same for x_2.

Closing the box again and again: a chain of four machines

$$
\begin{aligned}
g_4(x_4) &= \textstyle\sum_{x_1,x_2,x_3} f_{12}(x_1,x_2)\, f_{23}(x_2,x_3)\, f_{34}(x_3,x_4) \\
&= \textstyle\sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3) \sum_{x_1} f_{12}(x_1,x_2) \\
&= \textstyle\sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3)\, \ell_2(x_2)
\end{aligned}
$$

Close the first box, the blue bracket: call it $\ell_2(x_2)$, two numbers. X_1 and f_{12} are never looked at again.

Closing the box again and again: a chain of four machines

$$
\begin{aligned}
g_4(x_4) &= \sum_{x_1,x_2,x_3} f_{12}(x_1,x_2)\, f_{23}(x_2,x_3)\, f_{34}(x_3,x_4) \\
&= \sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3) \sum_{x_1} f_{12}(x_1,x_2) \\
&= \sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3)\, \ell_2(x_2) \\
&= \sum_{x_3} f_{34}(x_3,x_4)\, \ell_3(x_3)
\end{aligned}
$$

Close the second: $\ell_3(x_3)$, two numbers again. The box now hides three nodes.

Closing the box again and again: a chain of four machines

$$
\begin{aligned}
g_4(x_4) &= \textstyle\sum_{x_1,x_2,x_3} f_{12}(x_1,x_2)\, f_{23}(x_2,x_3)\, f_{34}(x_3,x_4) \\
&= \textstyle\sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3) \sum_{x_1} f_{12}(x_1,x_2) \\
&= \textstyle\sum_{x_3} f_{34}(x_3,x_4) \sum_{x_2} f_{23}(x_2,x_3)\, \ell_2(x_2) \\
&= \textstyle\sum_{x_3} f_{34}(x_3,x_4)\, \ell_3(x_3) \\
&= \ell_4(x_4)
\end{aligned}
$$

Close the third and read the answer. Three boxes, one per table. How much did each new machine cost?

Each new machine costs one more box; listing every case doubles

machines k	3	4	10	20

A chain of k machines, each 0 or 1, and $k - 1$ tables.

Each new machine costs one more box; listing every case doubles

machines k	3	4	10	20
cases to list, 2^k	8	16	1 024	1 048 576

Every new machine doubles the list: each old case splits into two.

Each new machine costs one more box; listing every case doubles

machines k	3	4	10	20
cases to list, 2^k	8	16	1 024	1 048 576
boxes to close, $k-1$	2	3	9	19

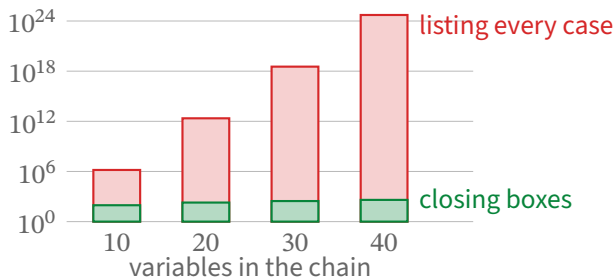
Every new machine adds one box, and a box never looks inside the boxes before it.

Each new machine costs one more box; listing every case doubles

machines k	3	4	10	20
cases to list, 2^k	8	16	1 024	1 048 576
boxes to close, $k-1$	2	3	9	19
numbers written, $2(k-1)$	4	6	18	38

Each box is two numbers, each number two products and one sum: six operations, however long the chain already is.

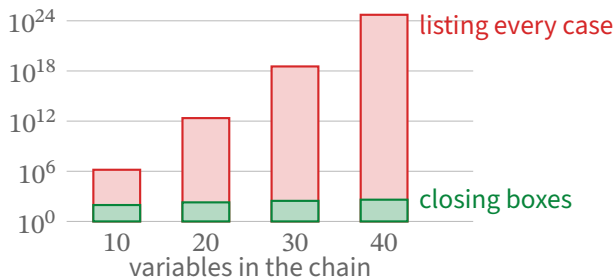
Listing every case doubles with each variable; closing boxes does not



2^k against a few $\times$ k

The same table, drawn: the red stack leaves the page by $k = 40$; the green one stays on the floor.

Listing every case doubles with each variable; closing boxes does not



2^k against a few $\times$ k

The lattice's 451 500 cells sit on the lower line: 1500 boxes of 301 numbers.
The linear promise of slide 2 is kept. What can go wrong?

A classmate sums the wrong variable away, and nothing complains

sum over x_1



$\times r = (1, 4)$

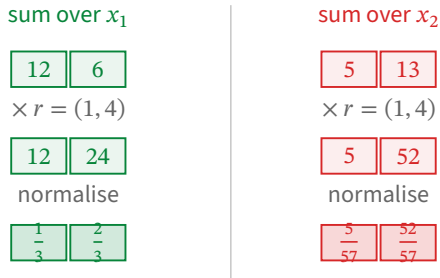
sum over x_2



$\times r = (1, 4)$

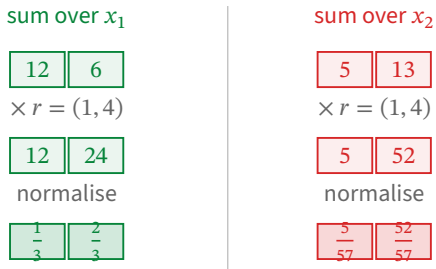
Closing the first box means summing x_1 away. A classmate sums x_2: $(4{+}1,\ 8{+}5) = (5, 13)$, the row instead of the column.

A classmate sums the wrong variable away, and nothing complains



Two numbers again, because $|\mathcal{X}_1| = |\mathcal{X}_2| = 2$. No error is raised.

A classmate sums the wrong variable away, and nothing complains



both are non-negative and both sum to one

It even normalises. A valid distribution, for the wrong question. What test would have caught it?

So what test could have caught it?

theirs | 5 | 52 |

Normalisation cannot. What property of the model would separate the two?

So what test could have caught it?

theirs | 5 | 52 |

Normalisation cannot. What property of the model would separate the two?

Checkpoint: Unit 1

Claim	Closing a box replaces a whole branch by two numbers, and each new variable costs one more box, not twice the cases.
Condition	The box and the rest share exactly one variable, the one on the edge.
Failure	Summing the wrong variable away still returns a valid distribution.
Next	A chain has one branch per side. A tree has several: where do the boxes go, and in what order?

2. The whole method on a tree: boxes become messages

Five machines and a branch. Where do the boxes go, and in what order?

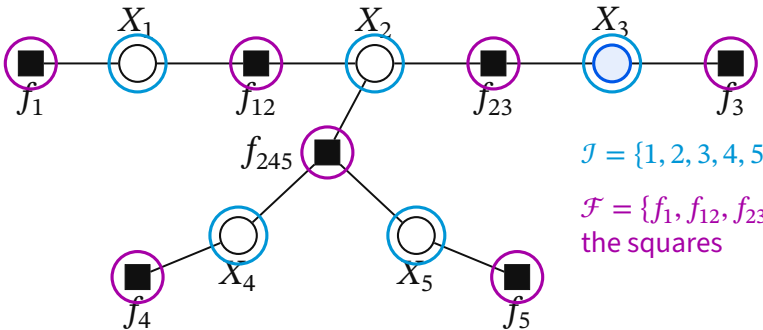
You will run the whole calculation on the tree and read off every share.

Where we are

1. Why: a loss that is a sum with 10^{338} terms
2. One step by hand: pull the shared factor out, close a box
3. **The whole method on a tree: every box becomes a list on an edge; two sweeps**
4. Why it is exact: the theorem, its conditions, how it breaks
5. The same theorem with $\max$; with a cycle; with continuous values
6. Where it is used

In hand: one box, closed by hand, and the promise that boxes repeat. This part closes every box of a tree, in an order that works.

Five machines now, and the same drawing with its parts named

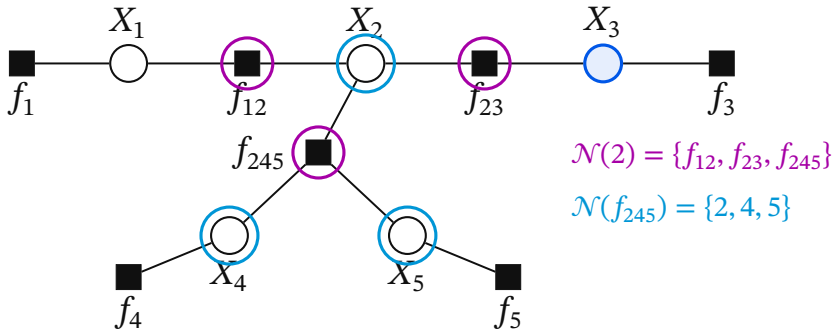


$\mathcal{I} = \{1, 2, 3, 4, 5\}$, the circles

$\mathcal{F} = \{f_1, f_{12}, f_{23}, f_3, f_{245}, f_4, f_5\}$, the squares

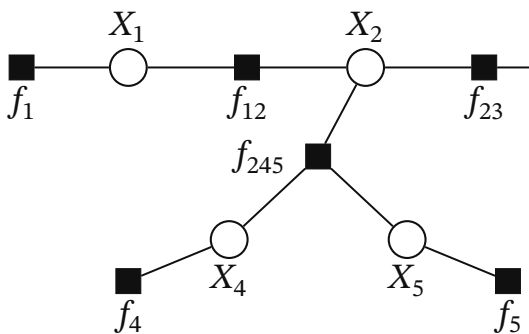
Machines 4 and 5 share the supply line, coupled by f_{245}. Each free end carries a table of ones, f_1, f_4, f_5, so that every list sent later has a source.

Five machines now, and the same drawing with its parts named



$\mathcal{N}(f)$: the variables f reads. $\mathcal{N}(i)$: the local functions that read X_i. Both are read off the edges.

Five machines now, and the same drawing with its parts named



last week → this week

$V \longrightarrow \mathcal{I}$

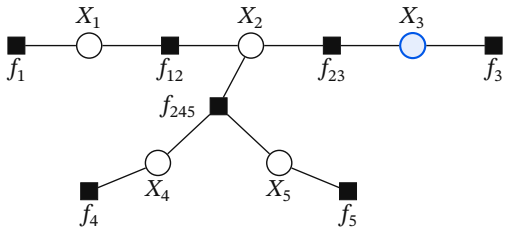
$F \longrightarrow \mathcal{F}$

$S_a \longrightarrow \mathcal{N}(f)$

$\prod_{a \in F} f_a \longrightarrow \prod_{f \in \mathcal{F}} f$

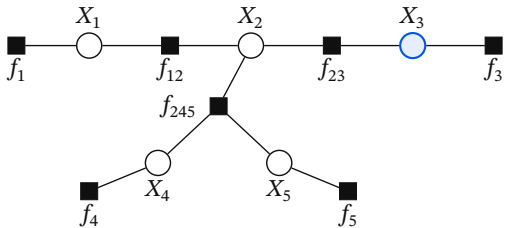
The same three sets, under last week's names. What shape is this drawing?

The drawing is a tree: eleven edges and no way round



Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

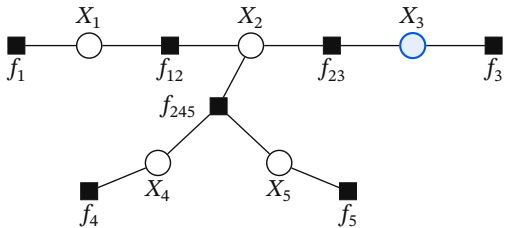
The drawing is a tree: eleven edges and no way round



Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

Cycle: a way round to the start. None here.

The drawing is a tree: eleven edges and no way round

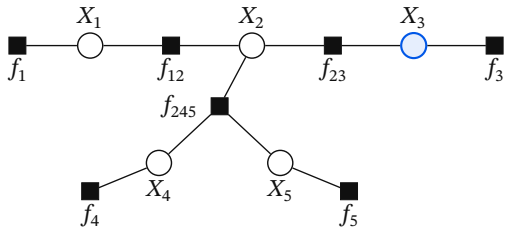


Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

Cycle: a way round to the start. None here.

Tree: all nodes connected, no cycle.

The drawing is a tree: eleven edges and no way round



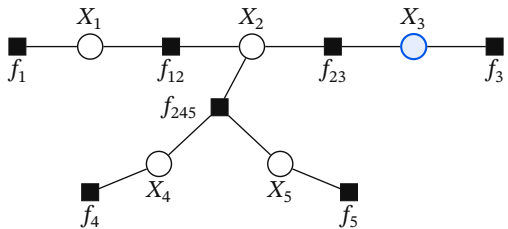
Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

Cycle: a way round to the start. None here.

Tree: all nodes connected, no cycle.

Leaf: a node with one edge: f_1, f_3, f_4, f_5.

The drawing is a tree: eleven edges and no way round



Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

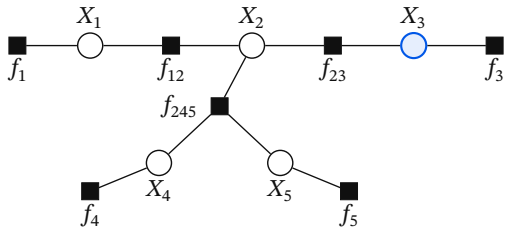
Cycle: a way round to the start. None here.

Tree: all nodes connected, no cycle.

Leaf: a node with one edge: f_1, f_3, f_4, f_5.

Root: the node we add up toward; today X_2.

The drawing is a tree: eleven edges and no way round



Edge: f to a variable it reads; eleven, the set $\mathcal{E}$.

Cycle: a way round to the start. None here.

Tree: all nodes connected, no cycle.

Leaf: a node with one edge: f_1, f_3, f_4, f_5.

Root: the node we add up toward; today X_2.

Branch: all that hangs off the root on one edge; three at X_2.

The seven tables, written out

f_{12}

x_1 \ x_2	0	1
0	4	1
1	8	5

f_{23}

x_2 \ x_3	0	1
0	3	1
1	1	4

$x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$

$x_2\,x_4$	low	normal	high
0 0	4	2	1
0 1	1	1	1
1 0	1	2	3
1 1	1	4	9

f_3		
	0	1

f_1		
	1	1

f_4		
	1	1

f_5			
	1	1	1

f_{245}: rows $x_2\,x_4$, each 0 or 1; columns $x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$, so $|\mathcal{X}_5| = 3$ and twelve entries. The grey tables are all ones.

The seven tables, written out

f_{12}

x_1 \ x_2	0	1
0	4	1
1	8	5

f_{23}

x_2 \ x_3	0	1
0	3	1
1	1	4

$x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$

x_2	x_4	low	normal	high
0	0	4	2	1
0	1	1	1	1
1	0	1	2	3
1	1	1	4	9

f_3 | 0 | 1 |

f_1 | 1 | 1 |

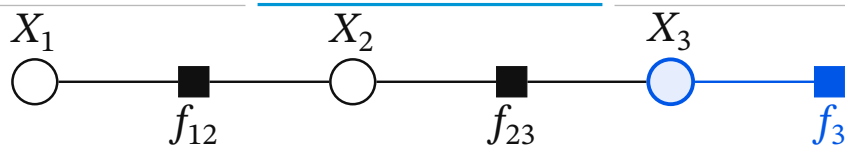
f_4 | 1 | 1 |

f_5 | 1 | 1 | 1 |

$$g(x) = f_1(x_1)\, f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)\, f_3(x_3)\, f_{245}(x_2, x_4, x_5)\, f_4(x_4)\, f_5(x_5)$$

One weight per assignment, 48 of them; the ones change none. And Machine 3's alert?

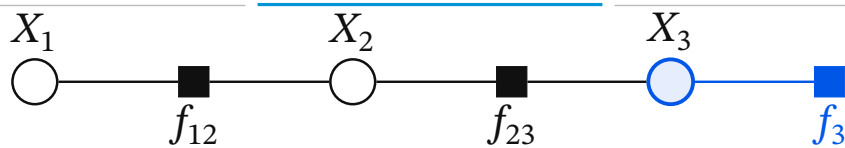
Machine 3's alert is one more table, not a special case



Definition 3 (evidence local function). Observing $X_i = c$ is recorded by $f_i(x_i) = 1$ when $x_i = c$ and 0 otherwise, multiplied into g.

Unit 1 did exactly this with $f_3 = (0, 1)$. A decoder that forbids a token uses the same table.

Machine 3's alert is one more table, not a special case



Definition 3 (evidence local function). Observing $X_i = c$ is recorded by $f_i(x_i) = 1$ when $x_i = c$ and 0 otherwise, multiplied into g.

A zero forbids a value. A sensor that says nothing is a table of ones, like f_1, not a zero. Now the target on this model.

The target on this model: the share of each x_2, given the alert

$p(x_2 \mid x_3{=}1)$: among assignments with $x_3 = 1$, the share holding each value of x_2.

The bar reads “given”: $x_3 = 1$ is fixed by the alert, so f_3 is already in the product.

The target on this model: the share of each x_2, given the alert

$p(x_2 \mid x_3{=}1)$: among assignments with $x_3 = 1$, the share holding each value of x_2.

Week 1's way: list all 48 assignments, sort by x_2, add each group.

Two group totals, $g_2(0)$ and $g_2(1)$, each a sum of 24 products of seven numbers.

The target on this model: the share of each x_2, given the alert

$p(x_2 \mid x_3{=}1)$: among assignments with $x_3 = 1$, the share holding each value of x_2.

Week 1's way: list all 48 assignments, sort by x_2, add each group.

Unit 1's way: write the group total, pull shared factors out.

Does the same move work with three branches?

The group total for either x_2, in two lines

$$g_2(x_2) = \sum_{x_1,x_3,x_4,x_5} f_1(x_1)\, f_{12}(x_1, x_2)\, f_{23}(x_2, x_3)\, f_3(x_3) \times f_{245}(x_2, x_4, x_5)\, f_4(x_4)\, f_5(x_5)$$

Every table is here, nothing summed yet: 24 terms for each value of x_2.

The group total for either x_2, in two lines

$$g_2(x_2) = \sum_{x_1,x_3,x_4,x_5} f_1(x_1)\, f_{12}(x_1,x_2)\, f_{23}(x_2,x_3)\, f_3(x_3) \times f_{245}(x_2,x_4,x_5)\, f_4(x_4)\, f_5(x_5)$$

$$= \underbrace{\Big(\sum_{x_1} \overbrace{f_1(x_1)}^{m_{f_1\to 1}}\, f_{12}(x_1,x_2)\Big)}_{m_{f_{12}\to 2}} \underbrace{\Big(\sum_{x_3} f_{23}(x_2,x_3)\, \overbrace{f_3(x_3)}^{m_{f_3\to 3}}\Big)}_{m_{f_{23}\to 2}}$$

$$\times \underbrace{\Big(\sum_{x_4,x_5} \overbrace{f_4(x_4)}^{m_{f_4\to 4}}\, \overbrace{f_5(x_5)}^{m_{f_5\to 5}}\, f_{245}(x_2,x_4,x_5)\Big)}_{m_{f_{245}\to 2}}$$

Lemma 1 three times: each other variable sits in one bracket only. Three brackets, three branches. Each bracket needs a name.

Each bracket on that line has a name

Definition 4 (message). Cut the edge between f and X_i; the tree falls into two sides. Let $\mathcal{F}_{f\to i}$ be the local functions on f's side and $\mathcal{I}_{f\to i}$ the variables on f's side other than i. The **message** from f to i is $m_{f\to i}(x_i) = \sum_{x_j,\, j\in\mathcal{I}_{f\to i}} \prod_{h\in\mathcal{F}_{f\to i}} h(x_{\mathcal{N}(h)})$, one entry per $x_i \in \mathcal{X}_i$; h runs over every local function on that side, f itself among them. The same with i and f exchanged defines $m_{i\to f}$.

In words: multiply every table on f's side, sum away every variable there except x_i. The box on f's side, closed; X_i is the **receiver**.

Each bracket on that line has a name

Definition 4 (message). Cut the edge between f and X_i; the tree falls into two sides. Let $\mathcal{F}_{f\to i}$ be the local functions on f's side and $\mathcal{I}_{f\to i}$ the variables on f's side other than i. The **message** from f to i is $m_{f\to i}(x_i) = \sum_{x_j,\, j\in\mathcal{I}_{f\to i}} \prod_{h\in\mathcal{F}_{f\to i}} h(x_{\mathcal{N}(h)})$, one entry per $x_i \in \mathcal{X}_i$; h runs over every local function on that side, f itself among them. The same with i and f exchanged defines $m_{i\to f}$.

Example: cut f_{12}–X_2. On f_{12}'s side: $\mathcal{F} = \{f_1, f_{12}\}, \mathcal{I} = \{1\}$, so $m_{f_{12}\to 2}(x_2) = \sum_{x_1} f_1(x_1) f_{12}(x_1, x_2) = (12, 6)$: Unit 1's ℓ.

Each bracket on that line has a name

Definition 4 (message). Cut the edge between f and X_i; the tree falls into two sides. Let $\mathcal{F}_{f\to i}$ be the local functions on f's side and $\mathcal{I}_{f\to i}$ the variables on f's side other than i. The **message** from f to i is $m_{f\to i}(x_i) = \sum_{x_j,\, j\in\mathcal{I}_{f\to i}} \prod_{h\in\mathcal{F}_{f\to i}} h(x_{\mathcal{N}(h)})$, one entry per $x_i \in \mathcal{X}_i$; h runs over every local function on that side, f itself among them. The same with i and f exchanged defines $m_{i\to f}$.

Example: cut f_{245}–X_2. On f_{245}'s side: $\mathcal{F} = \{f_4, f_5, f_{245}\}$, $\mathcal{I} = \{4, 5\}$. What are the three values?

The twelve-entry table collapses to two numbers

$x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$

$x_2\,x_4$	low	normal	high
0 0	4	2	1
0 1	1	1	1
1 0	1	2	3
1 1	1	4	9

The hardest one first. Rows: $x_2\,x_4$. Columns: the supply level x_5. Sum each half over its six (x_4, x_5) pairs; f_4 and f_5 multiply every entry by one.

The twelve-entry table collapses to two numbers

$x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$

$x_2\, x_4$	low	normal	high
0 0	4	2	1
0 1	1	1	1
1 0	1	2	3
1 1	1	4	9

$$m_{f_{245}\to 2}(0) = 4 + 2 + 1 + 1 + 1 + 1 = 10$$

The two rows with $x_2 = 0$, all six entries added.

The twelve-entry table collapses to two numbers

$x_5 \in \mathcal{X}_5 = \{\text{low}, \text{normal}, \text{high}\}$

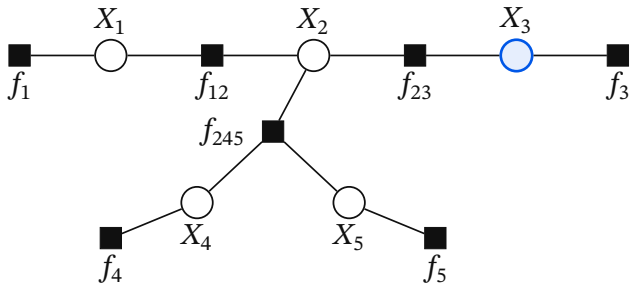
$x_2\,x_4$	low	normal	high
0 0	4	2	1
0 1	1	1	1
1 0	1	2	3
1 1	1	4	9

$$m_{f_{245}\to 2}(0) = 4 + 2 + 1 + 1 + 1 + 1 = 10$$

$$m_{f_{245}\to 2}(1) = 1 + 2 + 3 + 1 + 4 + 9 = 20$$

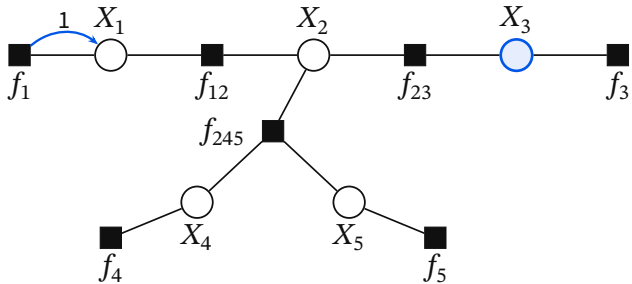
The two rows with $x_2 = 1$. So $m_{f_{245}\to 2} = (10, 20)$. Now all eleven arrows, on the drawing, in the order a machine can compute them.

Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



The three large brackets are the arrows into X_2, the small braces the leaves. Each is computed from arrows already drawn.

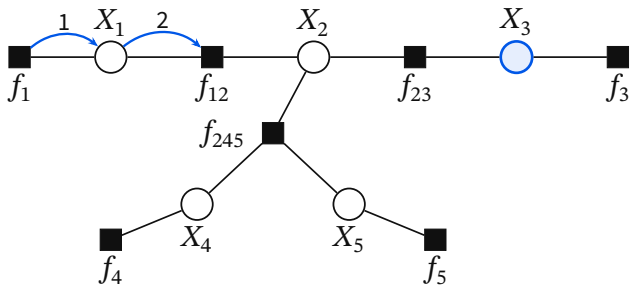
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



1: $m_{f_1 \to 1}(x_1) = f_1(x_1) = (1, 1)$

Definition 4 on the edge f_1–X_1: the far side is f_1 alone, nothing to sum. The small brace $m_{f_1 \to 1}$.

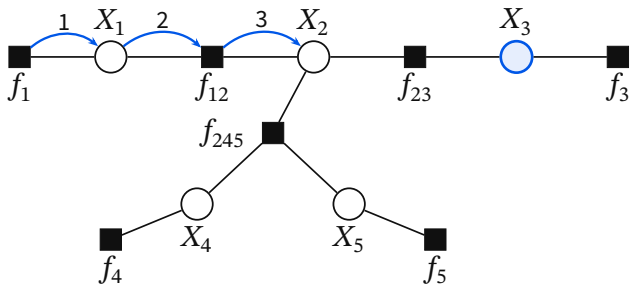
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



2: $m_{1\to f_{12}}(x_1) = m_{f_1\to 1}(x_1) = (1, 1)$, the only other arrival at X_1

Edge X_1–f_{12}: the far side is still just f_1, so the same list, passed on.

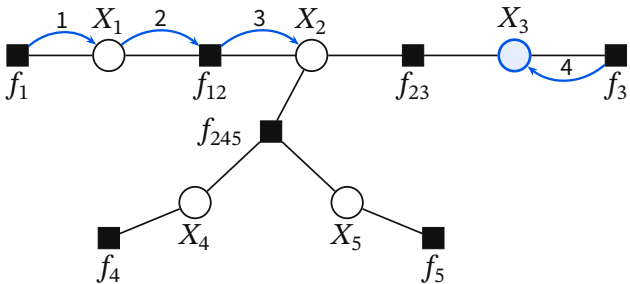
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



3: $m_{f_{12}\to 2}(x_2) = \sum_{x_1} m_{1\to f_{12}}(x_1)\, f_{12}(x_1, x_2) = (1{\cdot}4 + 1{\cdot}8,\ 1{\cdot}1 + 1{\cdot}5) = (12, 6)$

Edge f_{12}–X_2: far side f_1, X_1, f_{12}; sum x_1 away. The first large bracket, Unit 1's ℓ.

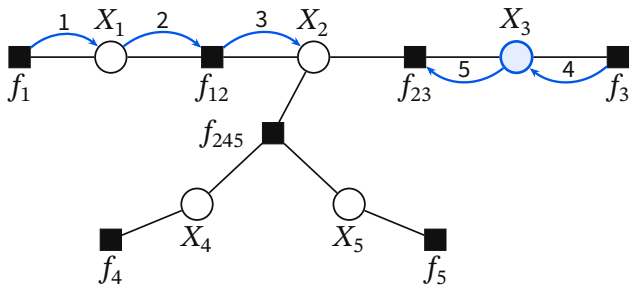
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



4: $m_{f_3 \to 3}(x_3) = f_3(x_3) = (0, 1)$

The evidence table, the small brace $m_{f_3 \to 3}$.

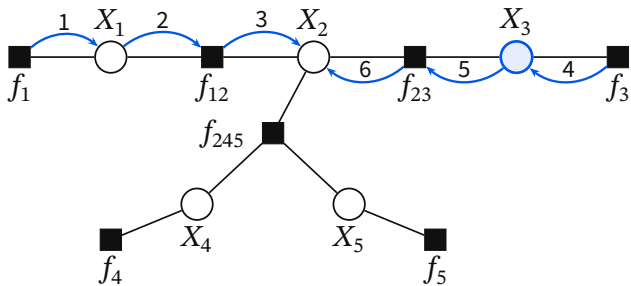
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



5: $m_{3\to f_{23}}(x_3) = m_{f_3\to 3}(x_3) = (0,1)$, the only other arrival at X_3

X_3 passes it on: the far side of X_3–f_{23} is still f_3 alone.

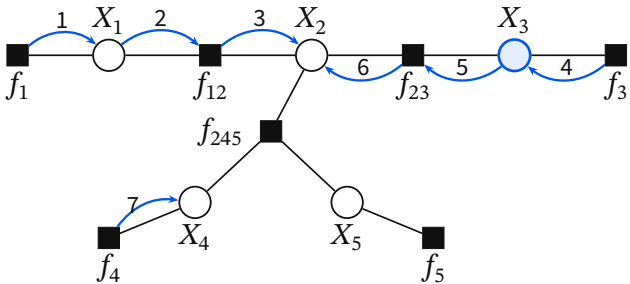
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



6: $m_{f_{23}\to 2}(x_2) = \sum_{x_3} f_{23}(x_2, x_3)\, m_{3\to f_{23}}(x_3) = (3{\cdot}0 + 1{\cdot}1,\ 1{\cdot}0 + 4{\cdot}1) = (1, 4)$

Edge f_{23}–X_2: sum x_3 away, the zero removing $x_3 = 0$. The second large bracket.

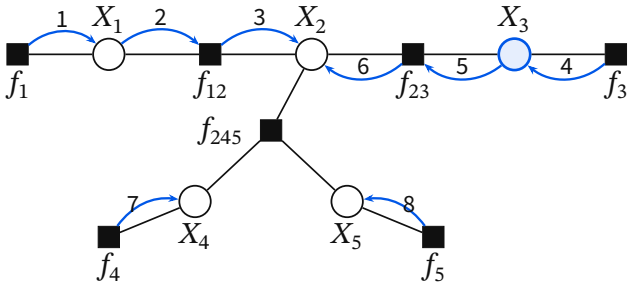
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



7: $m_{f_4 \to 4}(x_4) = f_4(x_4) = (1, 1)$

Another leaf table, the small brace $m_{f_4 \to 4}$.

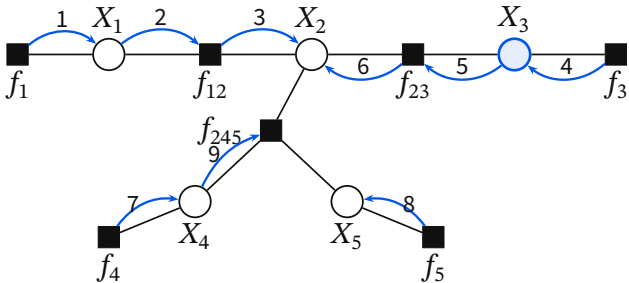
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



8: $m_{f_5 \to 5}(x_5) = f_5(x_5) = (1, 1, 1)$

Three ones, because X_5 has three values.

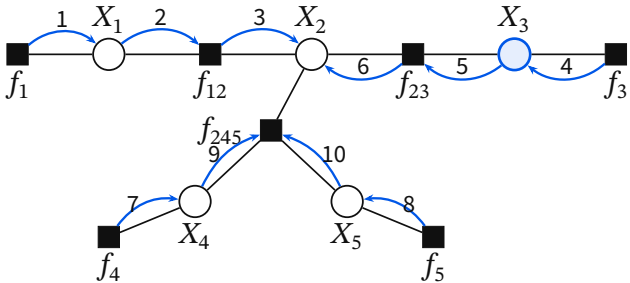
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



9: $m_{4\to f_{245}}(x_4) = m_{f_4\to 4}(x_4) = (1,1)$

X_4 passes its list on.

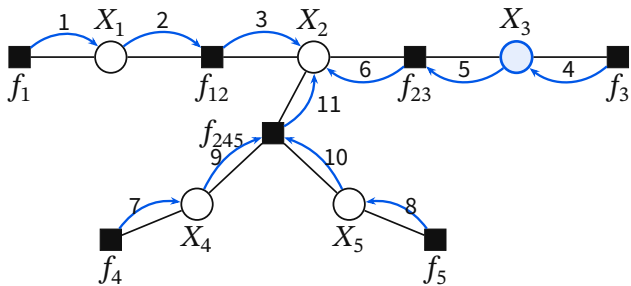
Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



10: $m_{5\to f_{245}}(x_5) = m_{f_5\to 5}(x_5) = (1,1,1)$

So does X_5.

Inward: the eleven arrows toward X_2, computed leaves first, on the drawing



11: $m_{f_{245}\to 2}(x_2) = \sum_{x_4,x_5} f_{245}(x_2, x_4, x_5)\, m_{4\to f_{245}}(x_4)\, m_{5\to f_{245}}(x_5) = (10, 20)$

Edge f_{245}–X_2: the twelve entries summed over x_4, x_5, the previous slide. All three have arrived: read the root.

Three lists have arrived at the root

$m_{f_{12}\to 2}$	12	6
$m_{f_{23}\to 2}$	1	4
$m_{f_{245}\to 2}$	10	20

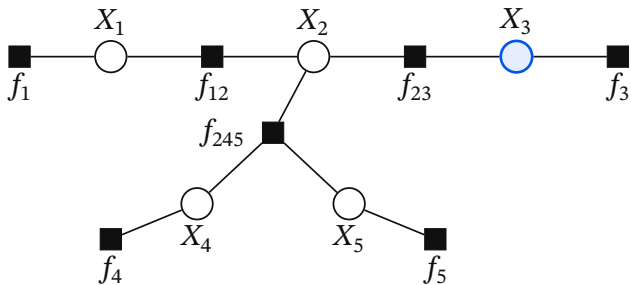
Multiply entry by entry. What is the total, and the share of each x_2?

Three lists have arrived at the root

$m_{f_{12}\to 2}$	12	6
$m_{f_{23}\to 2}$	1	4
$m_{f_{245}\to 2}$	10	20

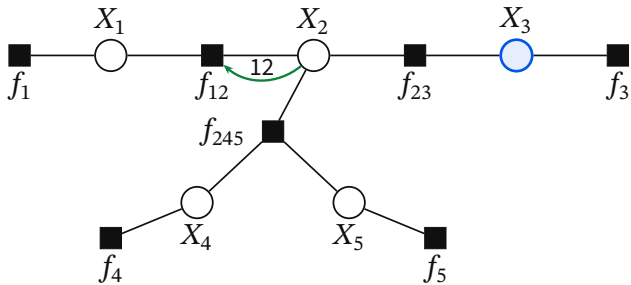
Multiply entry by entry. What is the total, and the share of each x_2?

Outward: eleven more messages, root first, and every variable becomes a root



To make X_1 the root we would close the boxes toward X_1. Most of them are already closed: reuse them.

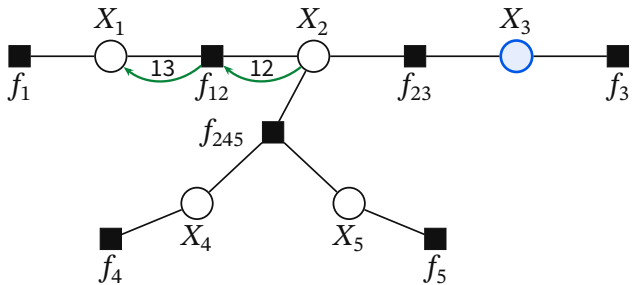
Outward: eleven more messages, root first, and every variable becomes a root



12: $m_{2\to f_{12}} = (1,4)\cdot(10,20) = (10,80)$

To f_{12}: everything that arrived at X_2 except what f_{12} itself sent.

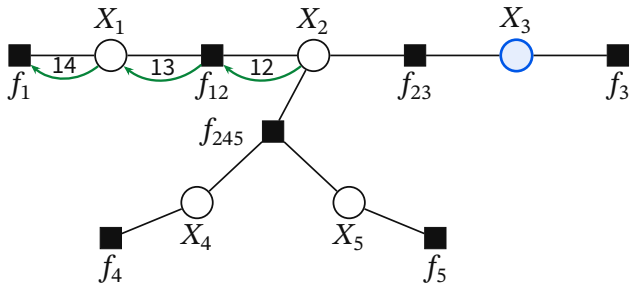
Outward: eleven more messages, root first, and every variable becomes a root



13: $m_{f_{12}\to 1} = (120, 480)$

f_{12} multiplies by its table and sums x_2 away. X_1 now holds its own group totals.

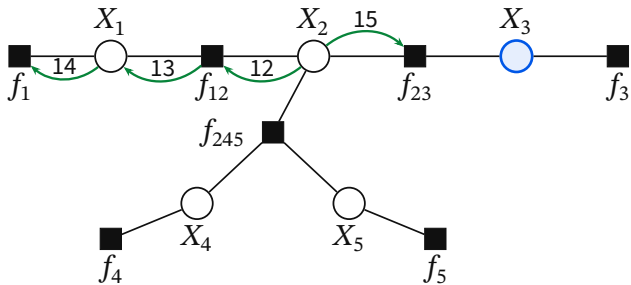
Outward: eleven more messages, root first, and every variable becomes a root



14: $m_{1 \to f_1} = (120, 480)$

X_1 passes it on to f_1; the edge now carries both directions.

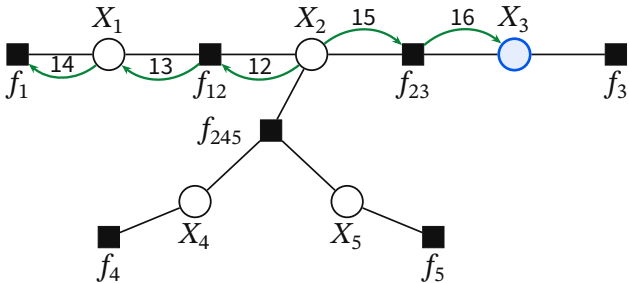
Outward: eleven more messages, root first, and every variable becomes a root



15: $m_{2\to f_{23}} = (12, 6) \cdot (10, 20) = (120, 120)$

Flat, although X_3 is certain: the alert is excluded on purpose, it belongs to the far side.

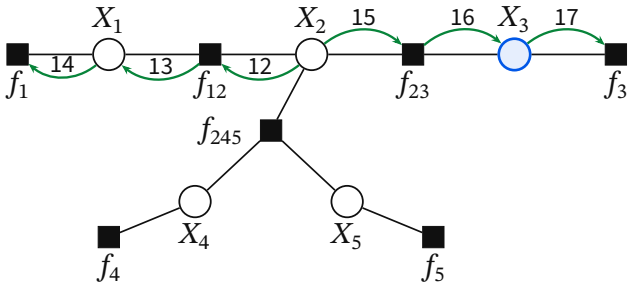
Outward: eleven more messages, root first, and every variable becomes a root



16: $m_{f_{23} \to 3} = (480, 600)$

Sum x_2 away through f_{23}.

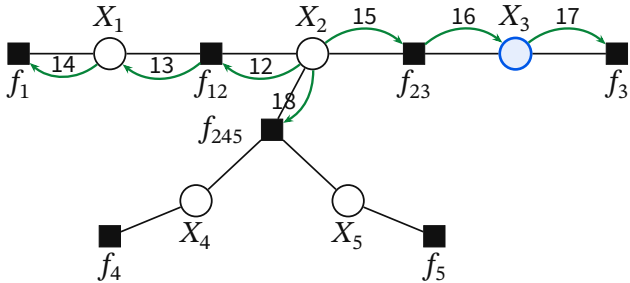
Outward: eleven more messages, root first, and every variable becomes a root



17: $m_{3 \to f_3} = (480, 600)$

X_3 forwards the list unchanged to f_3.

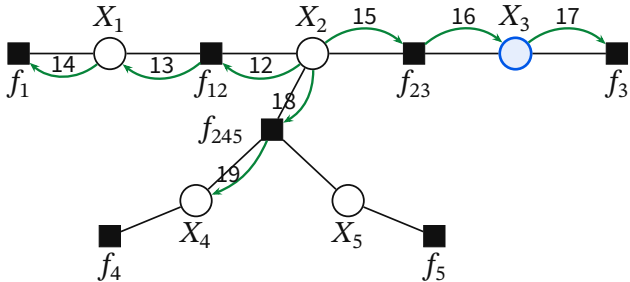
Outward: eleven more messages, root first, and every variable becomes a root



18: $m_{2 \to f_{245}} = (12, 6) \cdot (1, 4) = (12, 24)$

To the supply table: the other two brackets.

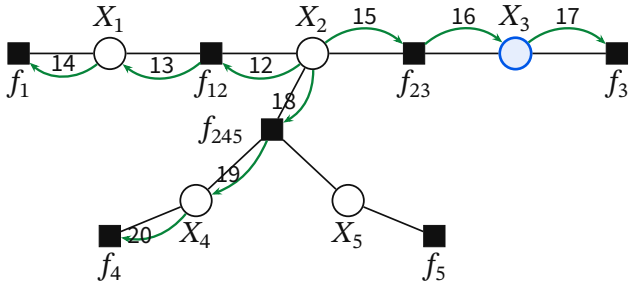
Outward: eleven more messages, root first, and every variable becomes a root



19: $m_{f_{245}\to 4} = (228, 372)$

Sum x_2 and x_5 away: two numbers for X_4.

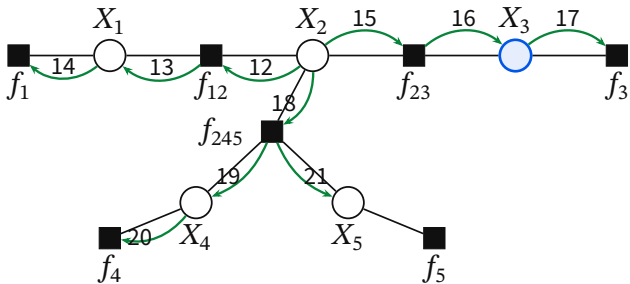
Outward: eleven more messages, root first, and every variable becomes a root



20: $m_{4 \to f_4} = (228, 372)$

X_4 forwards them to f_4.

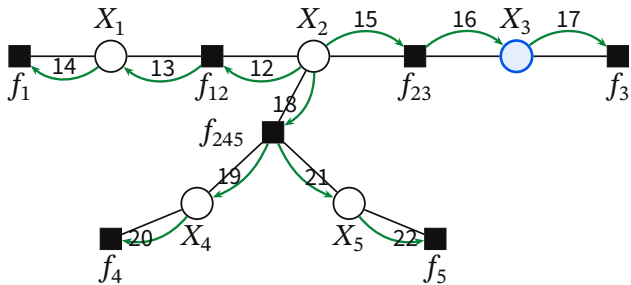
Outward: eleven more messages, root first, and every variable becomes a root



21: $m_{f_{245}\to 5} = (108, 180, 312)$

Sum x_2 and x_4 away: three numbers for X_5, one per supply level.

Outward: eleven more messages, root first, and every variable becomes a root



22: $m_{5\to f_5} = (108, 180, 312)$

X_5 forwards them to f_5. Twenty-two, complete. What does each variable now hold?

The product of everything arriving at a variable is its belief

Definition 5 (belief). The **belief** at variable i is $\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i)$, the product of every message arriving there.

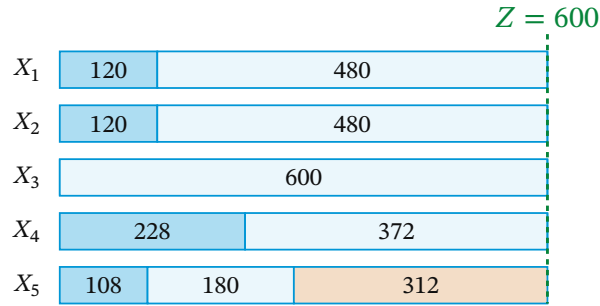
At X_2: $(12, 6) \cdot (1, 4) \cdot (10, 20) = (120, 480)$, the group totals. At X_1: $(1, 1) \cdot (120, 480)$.

The product of everything arriving at a variable is its belief

Definition 5 (belief). The **belief** at variable i is $\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i)$, the product of every message arriving there.

Not normalised, and the tilde says so: $\tilde{b}_i$ is a group total, not a share. The share is Definition 2's last step, $p(x_i) = \tilde{b}_i(x_i) / \sum_{x_i} \tilde{b}_i(x_i) = (120, 480)/600$. Do the five totals agree?

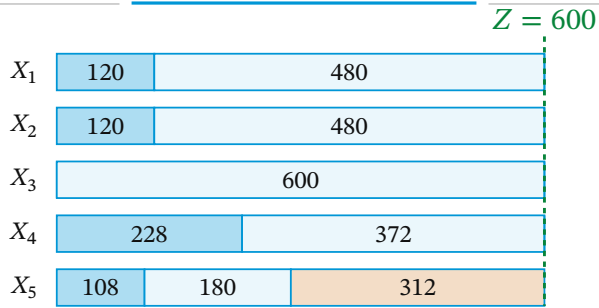
Every belief is a different split of one total



$$\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \qquad \sum_{x_i} \tilde{b}_i(x_i) = Z = 600$$

Five variables, five beliefs, one right-hand edge: each row adds to 600.

Every belief is a different split of one total



$$\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \qquad \sum_{x_i} \tilde{b}_i(x_i) = Z = 600$$

That is the partition function of Definition 2, found without listing the 48 assignments. Part 3 proves the rows must agree. What did the two sweeps buy?

What the two sweeps bought

One list per edge.

Twenty-two short lists, six of them plain copies, in place of 48 products of seven numbers.

What the two sweeps bought

One list per edge.

Computed once.

Each from its neighbours only, in an order where its inputs already exist.

What the two sweeps bought

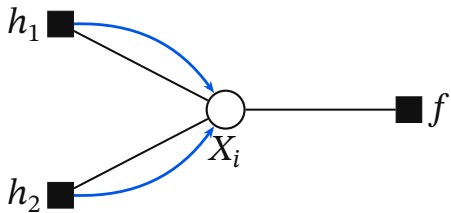
One list per edge.

Computed once.

Read twice.

Once on the way in, once on the way out, and every belief falls out of the products. Now the rules, written once.

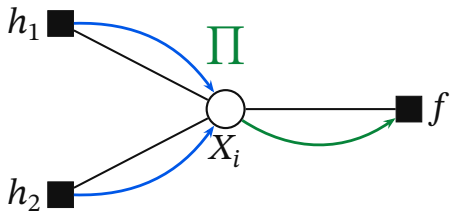
At a variable, the rule is a product



two messages arrive

Messages 2, 5, 9, 10 and 12, 15, 18 did this; with one neighbour to multiply, the product is a copy.

At a variable, the rule is a product

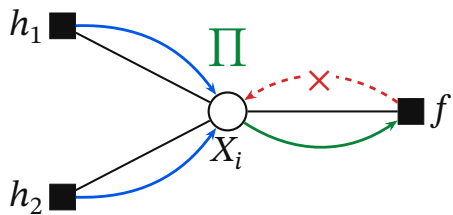


their product leaves

$$m_{i \to f}(x_i) = \prod_{h \in \mathcal{N}(i) \setminus \{f\}} m_{h \to i}(x_i)$$

$\mathcal{N}(i) \setminus \{f\}$ reads: every neighbour of i except f, the one being sent to.

At a variable, the rule is a product

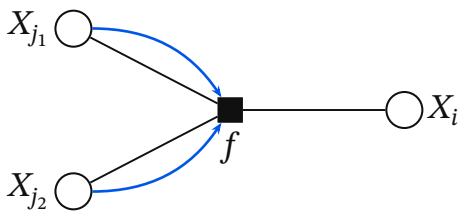


the message that came from f is left out

$$m_{i \to f}(x_i) = \prod_{h \in \mathcal{N}(i) \setminus \{f\}} m_{h \to i}(x_i)$$

The list f sent to X_i is not multiplied into the list X_i sends back to f: that would count f's table twice.

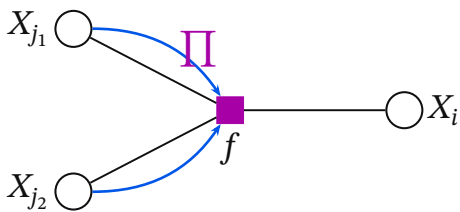
At a local function, multiply by the table, then sum the rest away



two messages arrive

Messages 3, 6, 11 and 13, 16, 19, 21 did this.

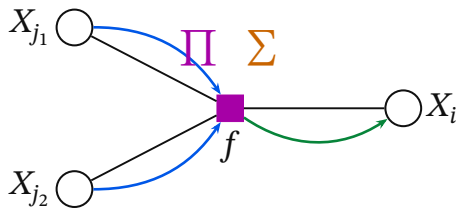
At a local function, multiply by the table, then sum the rest away



multiply by the table

The table is read once, multiplied by what arrived.

At a local function, multiply by the table, then sum the rest away

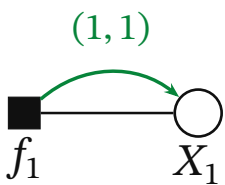


sum x_{j_1} and x_{j_2} away

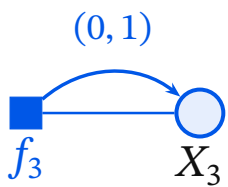
$$m_{f\to i}(x_i) = \sum_{x_{\mathcal{N}(f)\setminus\{i\}}} f(x_{\mathcal{N}(f)}) \prod_{j\in\mathcal{N}(f)\setminus\{i\}} m_{j\to f}(x_j)$$

Every variable f reads except i is summed away; only this rule removes a variable. And the leaves, which have no input?

Every leaf is a table, so every start is the same: send your table



a table of ones
changes nothing

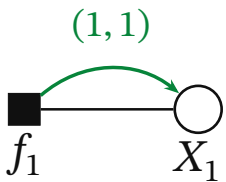


leaf local function
its own table

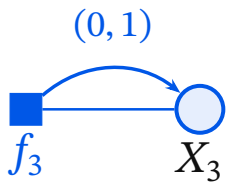
$m_{f \to i}(x_i) = f(x_i)$ when f reads X_i alone

The rule above with nothing to sum and nothing arriving: a table of ones sends ones, an evidence table sends its zero and its one.

Every leaf is a table, so every start is the same: send your table



a table of ones
changes nothing

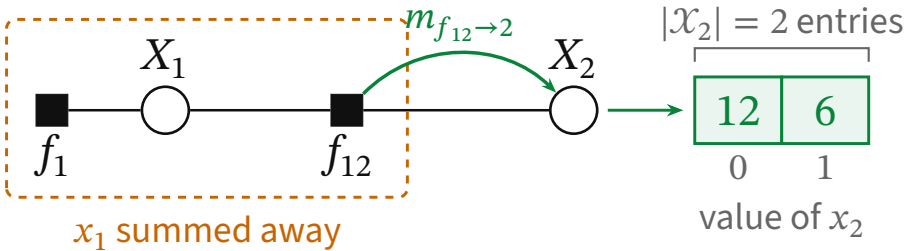


leaf local function
its own table

$m_{f \to i}(x_i) = f(x_i)$ when f reads X_i alone

Neither is a list of zeros. A list of zeros would kill every product it meets. How long is a message?

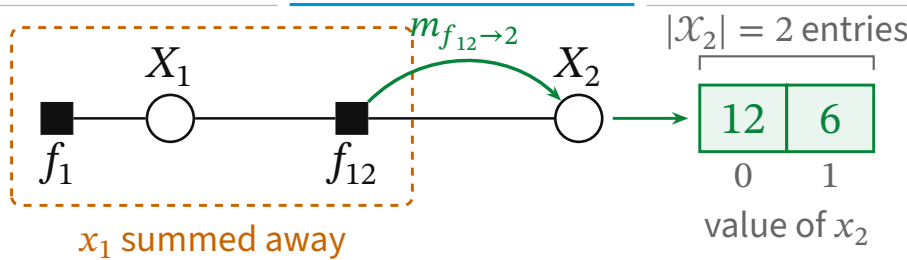
A message is a short list: one entry per value of the receiver



$$m_{f_{12}\to 2}(x_2) = \sum_{x_1} f_1(x_1)\, f_{12}(x_1, x_2)$$

Its length is $|\mathcal{X}_2|$, the receiver's number of values, whatever the table's size.

A message is a short list: one entry per value of the receiver



$$m_{f_{12}\to 2}(x_2) = \sum_{x_1} f_1(x_1)\, f_{12}(x_1, x_2)$$

The same twelve-entry table sent two numbers to X_4 and three to X_5. When is a message allowed to be computed?

In what order may the twenty-two be computed? Only when their inputs exist

message	reads	so it can be computed
3: $m_{f_{12}\to 2}$	message 2, $m_{1\to f_{12}}$	once message 2 exists

Message 3 is the local-function rule at f_{12}; its one input is message 2.

In what order may the twenty-two be computed? Only when their inputs exist

message	reads	so it can be computed
3: $m_{f_{12}\to 2}$	message 2, $m_{1\to f_{12}}$	once message 2 exists
12: $m_{2\to f_{12}}$	messages 6 and 11	only after the whole inward sweep

Message 12 is the variable rule at X_2 toward f_{12}; its inputs are the two other lists that reached X_2.

In what order may the twenty-two be computed? Only when their inputs exist

message	reads	so it can be computed
3: $m_{f_{12}\to 2}$	message 2, $m_{1\to f_{12}}$	once message 2 exists
12: $m_{2\to f_{12}}$	messages 6 and 11	only after the whole inward sweep
1: $m_{f_1\to 1}$	nothing	at once: every leaf is

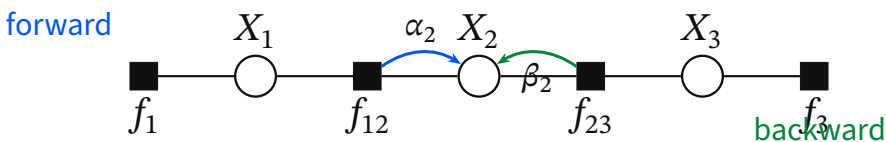
So a schedule always has somewhere to start.

In what order may the twenty-two be computed? Only when their inputs exist

Definition 6 (ready message and valid schedule). A message is **ready** when every message it reads already exists. A **valid schedule** computes messages only when they are ready, and is **complete** when all $2|\mathcal{E}| = 22$ exist, two per edge.

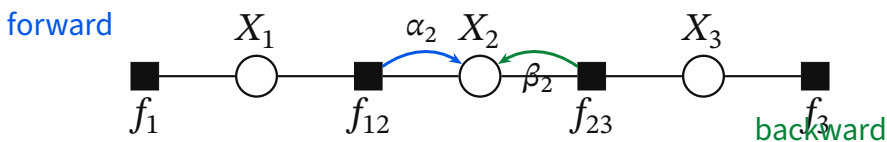
The two sweeps are one valid schedule. Part 3 shows that any other valid schedule gives the same 22 lists. These rules have older names.

On a chain these two rules already have names



A chain: a table between neighbours and one at each end. Each inner X_t has one edge on its left and one on its right.

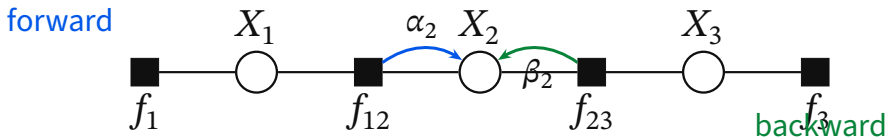
On a chain these two rules already have names



$\alpha_2 = m_{f_{12}\to 2}$, from the left;

The message from the left is the forward quantity $\alpha_t(s)$: the lattice cell of step 1, the total weight of every partial route reaching state s at step t.

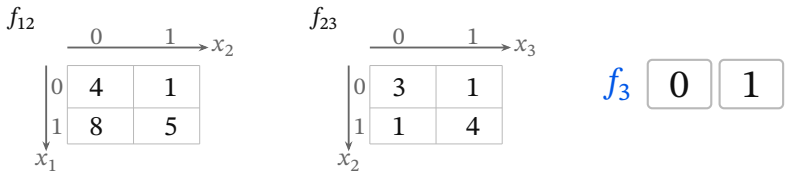
On a chain these two rules already have names



$\alpha_2 = m_{f_{12}\to 2}$, from the left; $\beta_2 = m_{f_{23}\to 2}$, from the right; $\tilde{b}_2 = \alpha_2\,\beta_2$

The message from the right is the backward quantity $\beta_t(s)$: the total weight of every way to finish from (t, s). Same two rules, older names. What can go wrong?

X_2 holds two lists and must send one on: which entries go into it?



arrived from the left, f_{12} : $m_{f_{12}\to 2} = (4{+}8,\ 1{+}5) = (12, 6)$

The three-machine chain with the alert. The column sums of f_{12} reached X_2 from the left.

X_2 holds two lists and must send one on: which entries go into it?

f_{12}

x_1 \ x_2	0	1
0	4	1
1	8	5

f_{23}

x_2 \ x_3	0	1
0	3	1
1	1	4

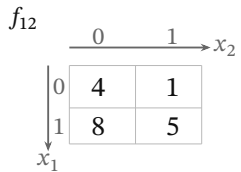
f_3 | 0 | 1 |

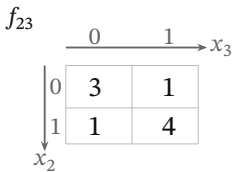
arrived from the left, f_{12} : $m_{f_{12}\to 2} = (4{+}8,\ 1{+}5) = (12, 6)$

arrived from the right, f_{23} : $m_{f_{23}\to 2} = (3{\cdot}0{+}1{\cdot}1,\ 1{\cdot}0{+}4{\cdot}1) = (1, 4)$

From the right, the $x_3{=}1$ column of f_{23}, kept by $f_3 = (0, 1)$.

X_2 holds two lists and must send one on: which entries go into it?





f_3 | 0 | 1 |

arrived from the left, f_{12} : $m_{f_{12}\to 2} = (4{+}8,\ 1{+}5) = (12, 6)$

arrived from the right, f_{23} : $m_{f_{23}\to 2} = (3{\cdot}0{+}1{\cdot}1,\ 1{\cdot}0{+}4{\cdot}1) = (1, 4)$

the rule sends to f_{12} : $m_{2\to f_{12}} = (1, 4)$

Now X_2 sends a list toward f_{12}. The rule: multiply what arrived, except the list that came from f_{12} itself. Only one list is left.

X_2 holds two lists and must send one on: which entries go into it?

f_{12}

x_1 \ x_2	0	1
0	4	1
1	8	5

f_{23}

x_2 \ x_3	0	1
0	3	1
1	1	4

f_3 | 0 | 1 |

arrived from the left, f_{12} : $m_{f_{12}\to 2} = (4{+}8,\ 1{+}5) = (12, 6)$

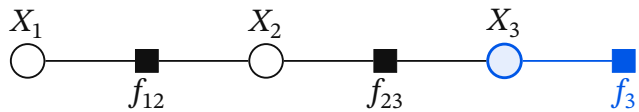
arrived from the right, f_{23} : $m_{f_{23}\to 2} = (3{\cdot}0{+}1{\cdot}1,\ 1{\cdot}0{+}4{\cdot}1) = (1, 4)$

the rule sends to f_{12} : $m_{2\to f_{12}} = (1, 4)$

a classmate sends to f_{12} : $m_{2\to f_{12}} = (12, 6) \cdot (1, 4) = (12, 24)$

The classmate forgets the exception and multiplies both lists. Two candidates. What does each do at X_1?

Both candidates reach X_1, and both normalise



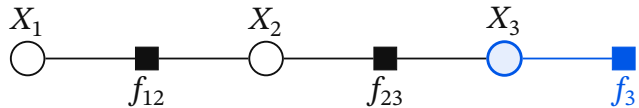
rule, $(1,4)$: $\tilde{b}_1(x_1) = \sum_{x_2} f_{12}(x_1, x_2)\, m_{2\to f_{12}}(x_2)$

$= (4{\cdot}1 + 1{\cdot}4,\ 8{\cdot}1 + 5{\cdot}4) = (8, 28)$

share $(8/36,\ 28/36) = (2/9,\ 7/9)$

f_{12} multiplies by its table and sums x_2 away: the rule of the previous slides, once more.

Both candidates reach X_1, and both normalise



rule, $(1, 4)$: $\tilde{b}_1(x_1) = \sum_{x_2} f_{12}(x_1, x_2)\, m_{2 \to f_{12}}(x_2)$

$= (4{\cdot}1 + 1{\cdot}4,\ 8{\cdot}1 + 5{\cdot}4) = (8, 28)$

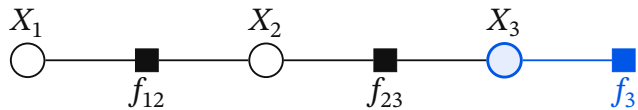
share $(8/36,\ 28/36) = (2/9,\ 7/9)$

classmate, $(12, 24)$: $\tilde{b}_1 = (4{\cdot}12 + 1{\cdot}24,\ 8{\cdot}12 + 5{\cdot}24) = (72, 216)$

share $(72/288,\ 216/288) = (1/4,\ 3/4)$

The same step on the classmate's list.

Both candidates reach X_1, and both normalise



rule, $(1,4)$: $\tilde{b}_1(x_1) = \sum_{x_2} f_{12}(x_1, x_2)\, m_{2\to f_{12}}(x_2)$

$= (4{\cdot}1 + 1{\cdot}4,\ 8{\cdot}1 + 5{\cdot}4) = (8, 28)$

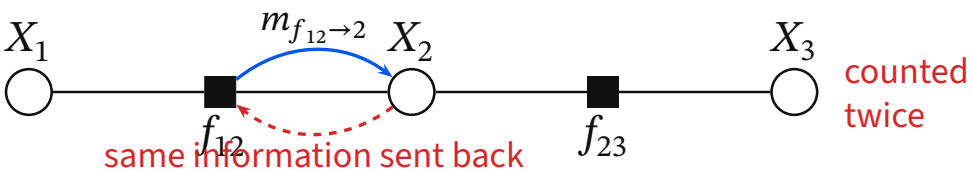
share $(8/36,\ 28/36) = (2/9,\ 7/9)$

classmate, $(12,24)$: $\tilde{b}_1 = (4{\cdot}12 + 1{\cdot}24,\ 8{\cdot}12 + 5{\cdot}24) = (72, 216)$

share $(72/288,\ 216/288) = (1/4,\ 3/4)$

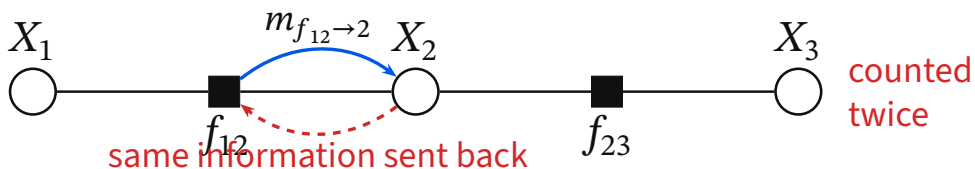
Two different shares, both non-negative, both adding to one. Which is right, and why?

The echoed message counts one table twice



$m_{f_{12}\to 2}$ already contains f_{12}; sending it back multiplies f_{12} in again.

The echoed message counts one table twice



Excluding the destination keeps every table in the product exactly once. And a zero in a table?

A zero does two different things



informative zero
the answer becomes certain

impossible observation
nothing to normalise

Suppose the model forbids a low supply while both machines alert: one belief entry becomes 0, and the share moves.

A zero does two different things

informative zero
the answer becomes certain



impossible observation
nothing to normalise

If every entry is 0, then $Z = 0$: the observation was impossible. An implementation must report this, not divide. One last look at that division.

You have normalised this sum since your first classifier

one softmax, over the vocabulary



$$Z = \sum_{v=1}^{50\,000} e^{s_v}$$

A softmax divides scores by their total.

You have normalised this sum since your first classifier

one softmax, over the vocabulary



$$Z = \sum_{v=1}^{50\,000} e^{s_v}$$

same object

2^{40} terms



$$Z = \sum_x \prod_f f$$

Here the scores are beliefs and the total is Z.

You have normalised this sum since your first classifier

one softmax, over the vocabulary



$$Z = \sum_{v=1}^{50\,000} e^{s_v}$$

same object

2^{40} terms



$$Z = \sum_x \prod_f f$$

on a tree: a few small tables, read twice

The difference is only how Z was found: read off a few small tables, twice.
Why is that exact?

Checkpoint: Unit 2

Claim Two sweeps over eleven edges produce every share, and Z.

Condition A message excludes whatever arrived from its destination.

Failure Echoing a message back returns a plausible distribution.

Next The rows agreed at 600. Step 4 proves they must: the theorem of slide 3.

Break, 10 minutes

Back for the proof: why two passes of local arithmetic return the exact marginal, and where that stops being true.

3. Why it is exact, how far it reaches, and what it is for

The five rows agreed at 600. Why must they, on every tree? How far does that reason reach, and where is it used?

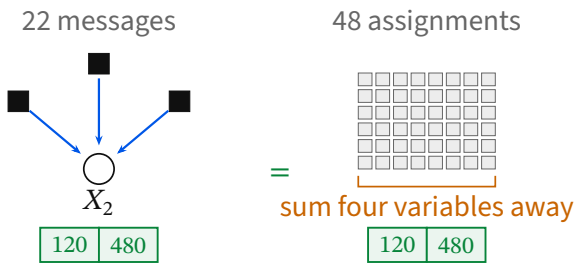
Steps 4, 5 and 6 of the map: prove it and break it; extend it to $\max$ and to a repaired cycle; return to the lattice of the opening.

Where we are

1. Why: a loss that is a sum with 10^{338} terms
2. One step by hand: pull the shared factor out, close a box
3. The whole method on a tree: every box becomes a list on an edge; two sweeps
4. **Why it is exact: the theorem, its conditions, how it breaks**
5. The same theorem with $\max$; with a cycle; with continuous values
6. Where it is used

In hand: twenty-two lists, two rules, and one model on which every belief matched brute force. This part turns that match into a theorem.

The claim, on the numbers we already have



At X_2 the product of the arriving lists, $(120, 480)$, is exactly the sum over 48 assignments, not close to it. Equality is why the speech model reports a loss, not an estimate. The theorem says this for every tree.

Theorem 1 states it for every variable of every tree

Theorem 1 (sum-product on a tree). Let the drawing be a tree. Compute all $2|\mathcal{E}|$ messages by the two rules, in any valid schedule, without rescaling. Then at every variable i, $\tilde{b}_i = g_i$ and $\sum_{x_i} \tilde{b}_i(x_i) = Z$.

On the running model: $\tilde{b}_2 = (120, 480) = g_2$, and $120 + 480 = 600 = Z$. The theorem says the same at $X_1, \dots, X_5$, and on any tree.

Theorem 1 states it for every variable of every tree

Theorem 1 (sum-product on a tree). Let the drawing be a tree. Compute all $2|\mathcal{E}|$ messages by the two rules, in any valid schedule, without rescaling. Then at every variable i, $\tilde{b}_i = g_i$ and $\sum_{x_i} \tilde{b}_i(x_i) = Z$.

Proof in three stages: the idea in one sentence; the proof carried out on our own tree, with the picture; then the general case, as two lemmas.

The idea of the proof, in one sentence

Cut an edge of a tree and the drawing falls into two parts that share one variable;

Nothing else is used. On a cycle, cutting one edge leaves the two parts still joined, and the sentence is false.

The idea of the proof, in one sentence

Cut an edge of a tree and the drawing falls into two parts that share one variable;

so the big sum splits into a product of two smaller sums (Lemma 1).

Unit 1 did this once, on a chain: $\ell \cdot r$.

The idea of the proof, in one sentence

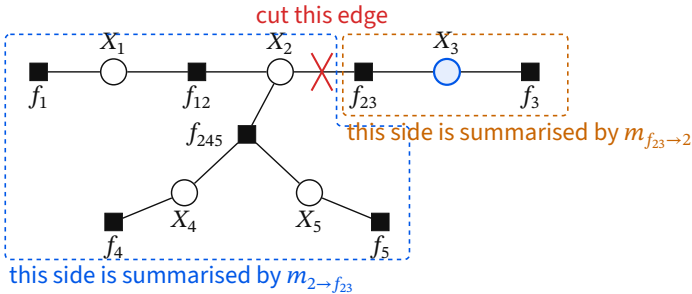
Cut an edge of a tree and the drawing falls into two parts that share one variable;

so the big sum splits into a product of two smaller sums (Lemma 1).

Cut again inside each part, down to the leaves: that is the two rules.

Watch it on our tree first, with the numbers.

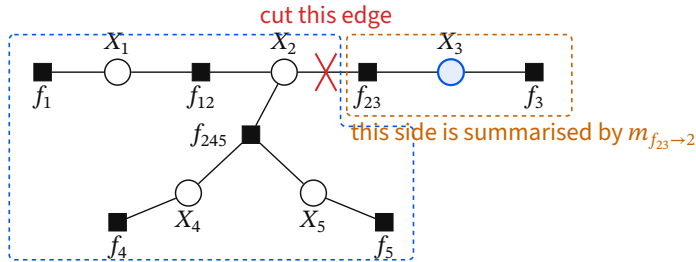
The proof on our tree: cut, split, cut again



$$g_2(x_2) = \sum_{x_1,x_3,x_4,x_5} f_1 f_{12}\, f_{23} f_3\, f_{245} f_4 f_5$$

The group total at X_2: twenty-four products of seven tables, for each x_2.

The proof on our tree: cut, split, cut again

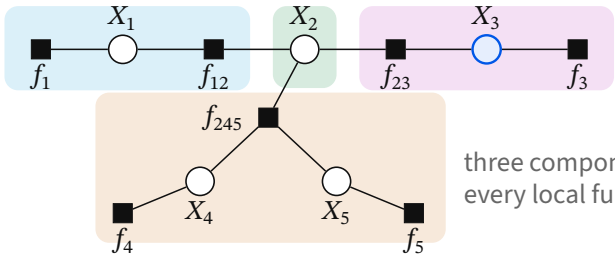


$$g_2(x_2) = \sum_{x_1,x_3,x_4,x_5} f_1 f_{12}\, f_{23} f_3\, f_{245} f_4 f_5$$

$$= \left(\sum_{x_1,x_4,x_5} f_1 f_{12} f_{245} f_4 f_5\right) \cdot \left(\sum_{x_3} f_{23} f_3\right)$$

Cut X_2–f_{23}: the sides share only x_2, so Lemma 1 splits the sum. Blue: near side. Orange: far side, $m_{f_{23}\to 2} = (1, 4)$.

The proof on our tree: cut, split, cut again

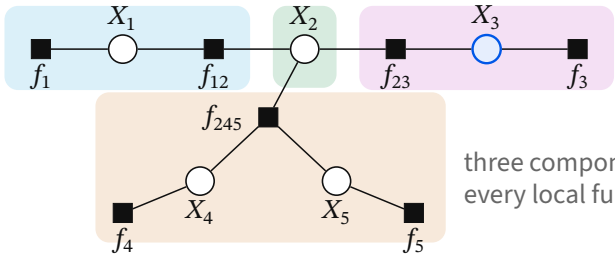


three components and one node, every local function in exactly one

$$g_2(x_2) = \left(\sum_{x_1} f_1 f_{12}\right)\left(\sum_{x_4,x_5} f_{245} f_4 f_5\right)\left(\sum_{x_3} f_{23} f_3\right) = (12, 6) \cdot (10, 20) \cdot (1, 4)$$

Cut the other two edges at X_2 the same way: three boxes, the three messages that arrived at X_2. Their product is $(120, 480) = g_2$. That is Theorem 1 at X_2.

The proof on our tree: cut, split, cut again

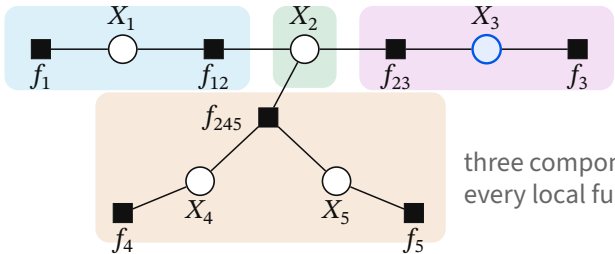


three components and one node, every local function in exactly one

$$g_2(x_2) = \left(\sum_{x_1} f_1 f_{12}\right)\left(\sum_{x_4,x_5} f_{245} f_4 f_5\right)\left(\sum_{x_3} f_{23} f_3\right) = (12, 6) \cdot (10, 20) \cdot (1, 4)$$

Inside the first box, cut X_1–f_1 once more: $\sum_{x_1} f_{12}\,(\sum f_1) = \sum_{x_1} f_{12}\, m_{f_1 \to 1}$, which is the local-function rule. Cutting inside a box is the rules.

The proof on our tree: cut, split, cut again



three components and one node, every local function in exactly one

$$g_2(x_2) = \left(\sum_{x_1} f_1 f_{12}\right)\left(\sum_{x_4,x_5} f_{245} f_4 f_5\right)\left(\sum_{x_3} f_{23} f_3\right) = (12, 6) \cdot (10, 20) \cdot (1, 4)$$

Two moves, then: cut at the root, cut inside each box. The general case names them Lemma 2 and Lemma 3.

General case, move 1 (Lemma 2): cut at any variable of any tree

Lemma 2 (the messages multiply). On a tree, with $m_{f \to i}$ the message of Definition 4, the box on f's side closed, $g_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i)$ for every variable i.

Cut every edge at X_i: each neighbour f keeps one side, and its message is that side summed with x_i fixed.

General case, move 1 (Lemma 2): cut at any variable of any tree

Lemma 2 (the messages multiply). On a tree, with $m_{f \to i}$ the message of Definition 4, the box on f's side closed, $g_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i)$ for every variable i.

Example at X_2: three sides, $(12, 6) \cdot (1, 4) \cdot (10, 20) = (120, 480) = g_2$.

General case, move 1 (Lemma 2): cut at any variable of any tree

Lemma 2 (the messages multiply). On a tree, with $m_{f \to i}$ the message of Definition 4, the box on f's side closed, $g_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i)$ for every variable i.

Proof on the next slide but one. First the second lemma.

General case, move 2 (Lemma 3): cut inside each box, down to the leaves

Lemma 3 (the rules compute the messages). On a tree, the messages of Definition 4 obey Unit 2's three rules: a leaf sends its table; a variable sends the product of the other arrivals; a local function multiplies by its table and sums the rest away. So the two sweeps deliver exactly those messages, all $2|\mathcal{E}|$ of them.

Example: the rule at f_{23} produced $(1, 4)$, and the box on f_{23}'s side, summed by hand, is $(1, 4)$. Twenty-two such equalities.

General case, move 2 (Lemma 3): cut inside each box, down to the leaves

Lemma 3 (the rules compute the messages). On a tree, the messages of Definition 4 obey Unit 2's three rules: a leaf sends its table; a variable sends the product of the other arrivals; a local function multiplies by its table and sums the rest away. So the two sweeps deliver exactly those messages, all $2|\mathcal{E}|$ of them.

Proof after the theorem. With both lemmas in hand:

Theorem 1 from the two lemmas, in one line

$$\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \quad \text{Definition 5}$$

The belief is the product of what the sweeps delivered.

Theorem 1 from the two lemmas, in one line

$$\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \quad \text{Definition 5}$$

$$= \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \quad \text{Lemma 3}$$

The same product, now read as Definition 4's boxes, because Lemma 3 says the two agree.

Theorem 1 from the two lemmas, in one line

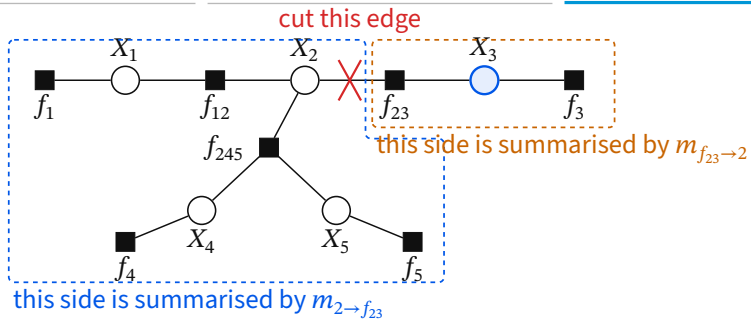
$$\tilde{b}_i(x_i) = \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \quad \text{Definition 5}$$

$$= \prod_{f \in \mathcal{N}(i)} m_{f \to i}(x_i) \quad \text{Lemma 3}$$

$$= g_i(x_i) \quad \text{Lemma 2}$$

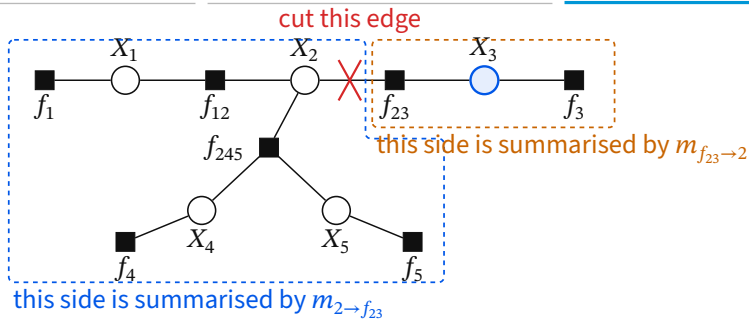
And the product of the boxes is the group total. Sum over x_i: $\sum_{x_i} g_i(x_i) = Z$, so every row adds to Z. Corollary: Z from any single belief, no extra pass. Now the two proofs.

Proof of Lemma 2: cut one edge, and the two sides share one variable



Cut X_2–f_{23}. A local function on one side cannot read a variable on the other: that would need the cut edge. The sides share only x_2.

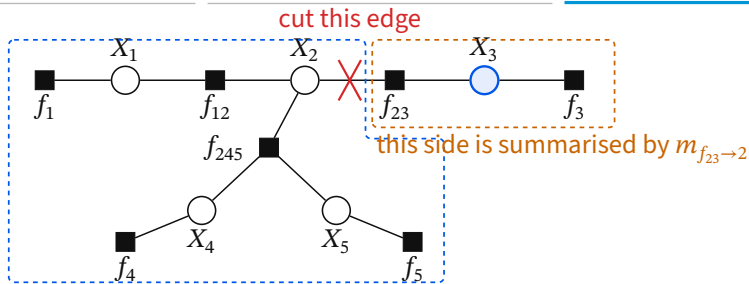
Proof of Lemma 2: cut one edge, and the two sides share one variable



$$g_2(x_2) = \sum_{\text{near side}} \sum_{\text{far side}} \left(\prod_{\text{near}} h\right)\left(\prod_{\text{far}} h\right) = m_{2\to f_{23}}(x_2) \cdot m_{f_{23}\to 2}(x_2)$$

So the product splits into a near factor and a far factor with no summation variable in common: Lemma 1, once. Each factor is a message of Definition 4.

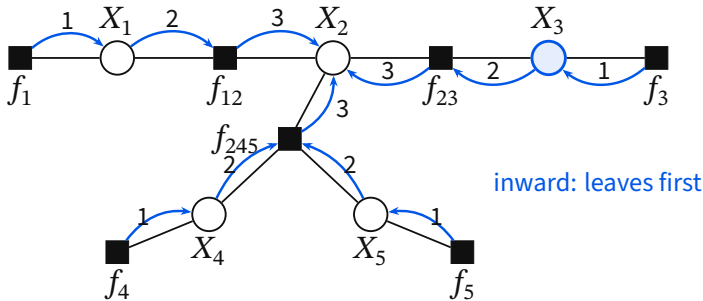
Proof of Lemma 2: cut one edge, and the two sides share one variable



$$g_2(x_2) = \sum_{\text{near side}} \sum_{\text{far side}} \left(\prod_{\text{near}} h\right)\left(\prod_{\text{far}} h\right) = m_{2\to f_{23}}(x_2) \cdot m_{f_{23}\to 2}(x_2)$$

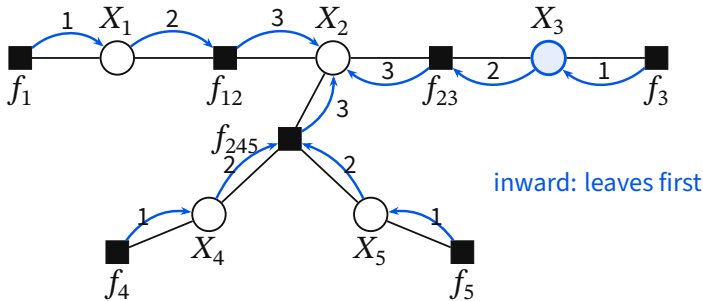
Cut the other edges at X_2 in turn: three sides, three messages. No cycle means the sides never rejoin. Lemma 2 is proved. Now Lemma 3.

Proof of Lemma 3: visit the messages in the order they were computed



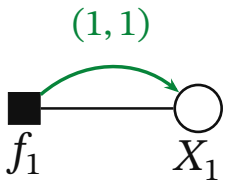
The numbers are depths: how many messages each one waited for. Show that Definition 4's message on each edge is what the rule gives, assuming that holds for the ones it reads.

Proof of Lemma 3: visit the messages in the order they were computed

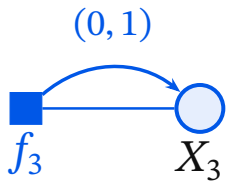


Depth 1 first: the leaves. Then a message at a variable, then one at a local function. Three cases, one slide each: the order in which the forward pass fills the lattice's columns.

Case 1, a leaf: the box on its side is the table itself



a table of ones
changes nothing

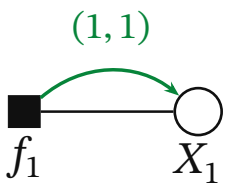


leaf local function
its own table

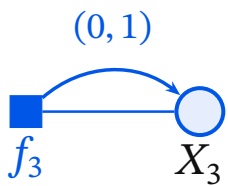
$$m_{f \to i}(x_i) = f(x_i)$$

Cut the edge at a leaf f: on f's side there is f alone and nothing to sum, so Definition 4 gives f's table. The rule sends f's table. Equal.

Case 1, a leaf: the box on its side is the table itself



a table of ones
changes nothing

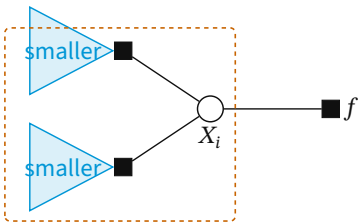


leaf local function
its own table

$$m_{f \to i}(x_i) = f(x_i)$$

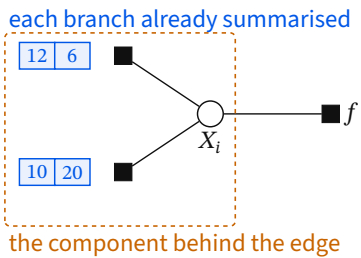
Example: $m_{f_3 \to 3} = (0, 1) = f_3$. The same at f_1, f_4, f_5.

Case 2, at a variable: the box is a product of smaller boxes



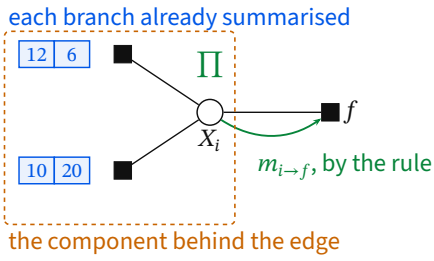
The box on X_i's side, away from f, is made of the branches at X_i other than f.

Case 2, at a variable: the box is a product of smaller boxes



Those branches share only x_i, so the box is the product of the branches' boxes, the messages $m_{h \to i}$, each on a smaller side.

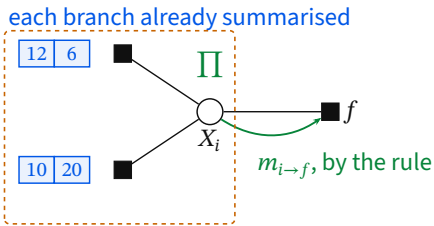
Case 2, at a variable: the box is a product of smaller boxes



$$m_{i\to f}(x_i) = \prod_{h\in\mathcal{N}(i)\setminus\{f\}} m_{h\to i}(x_i)$$

That is exactly the variable rule, applied to messages the sweep already holds. Equal.

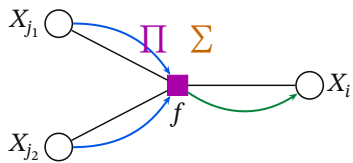
Case 2, at a variable: the box is a product of smaller boxes



$$m_{i\to f}(x_i) = \prod_{h\in\mathcal{N}(i)\setminus\{f\}} m_{h\to i}(x_i)$$

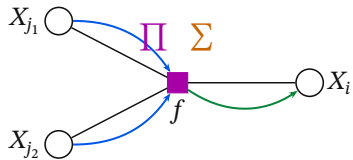
Example: $m_{2\to f_{12}} = (1,4)\cdot(10,20) = (10,80)$; the box on X_2's side away from f_{12}, summed by hand, is $(10,80)$.

Case 3, at a local function: multiply by the table, sum the rest away



The box on f's side is f itself and one branch per other variable f reads.

Case 3, at a local function: multiply by the table, sum the rest away

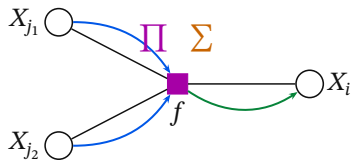


sum x_{j_1} and x_{j_2} away

$$m_{f\to i}(x_i) = \sum_{x_{\mathcal{N}(f)\setminus\{i\}}} f(x_{\mathcal{N}(f)}) \prod_{j\in\mathcal{N}(f)\setminus\{i\}} m_{j\to f}(x_j)$$

f reads only the variables on its own edges, so it comes outside every inner sum; each branch's box is Definition 4's $m_{j\to f}$, on a smaller side. That is the local-function rule. Equal.

Case 3, at a local function: multiply by the table, sum the rest away



sum x_{j_1} and x_{j_2} away

$$m_{f\to i}(x_i) = \sum_{x_{\mathcal{N}(f)\setminus\{i\}}} f(x_{\mathcal{N}(f)}) \prod_{j\in\mathcal{N}(f)\setminus\{i\}} m_{j\to f}(x_j)$$

Example: $m_{f_{245}\to 2} = \sum_{x_4,x_5} f_{245}\,(1,1)(1,1,1) = (10,20)$. All three cases hold, so the sweeps deliver all twenty-two messages of Definition 4. Lemma 3 is proved. Did the order matter?

Proposition 1: neither the order nor the root changes a single number

Proposition 1 (schedule independence). Every valid schedule computes the same $2|\mathcal{E}|$ values, and the root can be any node.

Definition 4 fixes each message by its side of the edge, not by when it was computed.

Proposition 1: neither the order nor the root changes a single number

Proposition 1 (schedule independence). Every valid schedule computes the same $2|\mathcal{E}|$ values, and the root can be any node.

Sweeping from X_4 changes no list; on a chain, forward-then-backward and the reverse agree. Where did the proof use the tree?

Where did the proof use that the drawing is a tree?

Look back at the two proofs. How many times was the tree shape used, and for what?

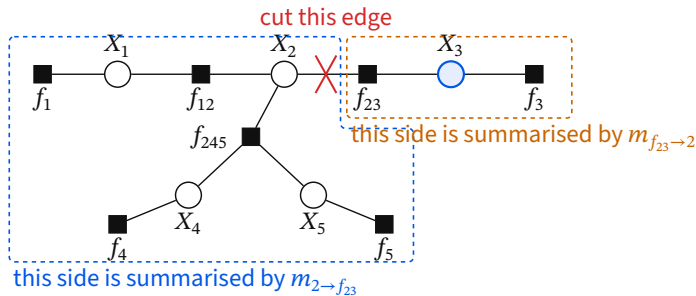
Predict before calculating.

Where did the proof use that the drawing is a tree?

Look back at the two proofs. How many times was the tree shape used, and for what?

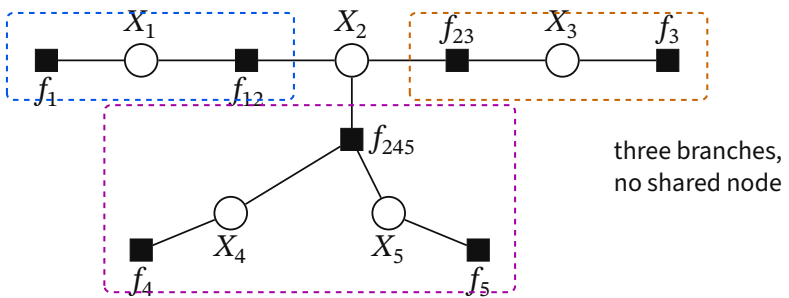
Predict before calculating.

The three places, drawn, and what a cycle would break



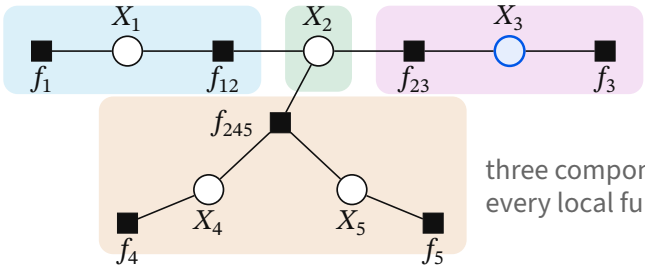
(1) Cut X_2–f_{23}: the left part and the right part have no node in common, so they share only x_2, and Lemma 1 splits the sum. On a cycle, the two sides would still be joined the other way round: no split.

The three places, drawn, and what a cycle would break



(2) At X_2 the three branches meet only at X_2. On a cycle, two branches would meet again elsewhere, share a variable, and the sums could no longer be moved inside the product.

The three places, drawn, and what a cycle would break



three components and one node, every local function in exactly one

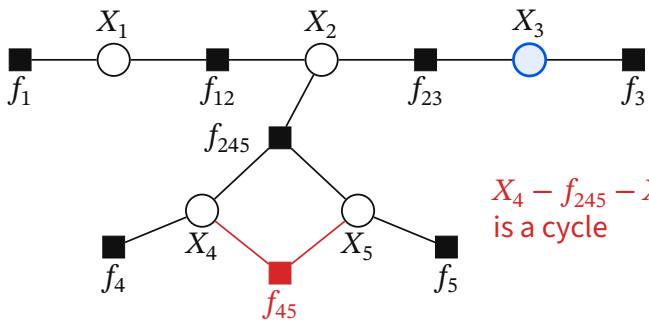
(3) The three branches together hold every table exactly once, so multiplying their three boxes uses each table once. On a cycle a table could sit in two branches and be counted twice: the classmate's error of Unit 2.

The theorem has three conditions; test each by breaking it

condition in Theorem 1	what we do with it now
the drawing is a tree	add one table and make a cycle
every message by the rules	keep, and ask what it costs
without rescaling	divide as we go, and see what changes

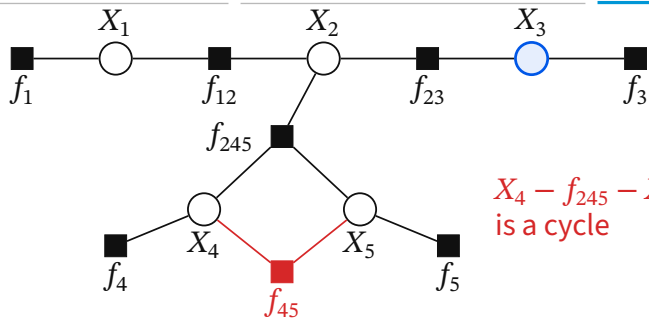
The speech lattice is a chain, a tree: the CTC loss is exact. First break the tree ourselves.

Condition 1 broken: one extra local function, and the tree is gone



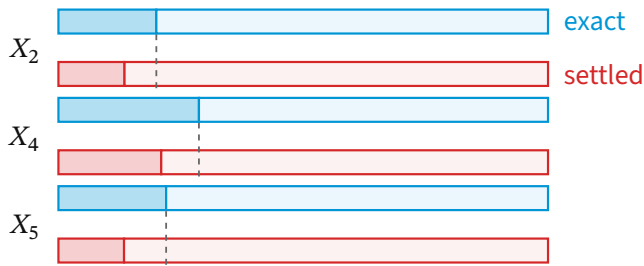
Add f_{45}, coupling Machines 4 and 5 directly. X_4–f_{245}–X_5–f_{45}–X_4 is a cycle.

Condition 1 broken: one extra local function, and the tree is gone



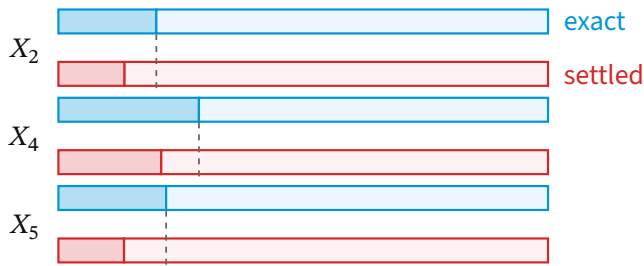
No edge on that cycle separates anything, so Lemma 2 fails; and no message on it is ever ready, so the schedule never starts. What if we run the updates anyway?

Run the rules on the cycle anyway: they settle, on the wrong answer



What is drawn: for X_2, X_4, X_5, one bar each; the filled part is the share of the value 0.

Run the rules on the cycle anyway: they settle, on the wrong answer



Blue, the exact share: list the 24 assignments that agree with the observation and add, as in Week 1.

Run the rules on the cycle anyway: they settle, on the wrong answer



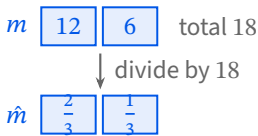
Red: start every list at ones, apply the two rules round after round along the cycle until no list changes any more, and read the beliefs.

Run the rules on the cycle anyway: they settle, on the wrong answer



They stop changing, but at 0.13 where the truth is 0.20, at 0.21 for 0.29, at 0.13 for 0.22. Convergence is not correctness; Week 6 asks what the settled numbers do minimise. Second condition: “without rescaling”.

Condition 3 broken: divide a list by its total, and write the total down



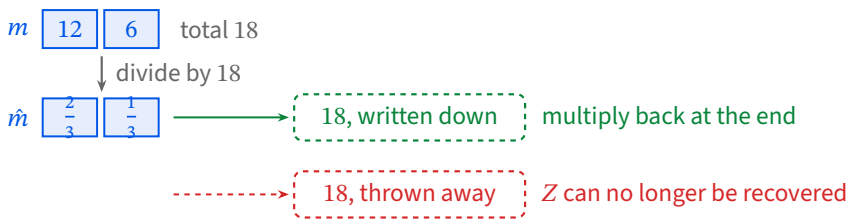
Long chains multiply many small numbers, so libraries divide each list by its total as they go. Take $m_{f_{12}\to 2} = (12, 6)$, total 18: divided, $(2/3, 1/3)$. Every list computed from it later is 18 times smaller than before.

Condition 3 broken: divide a list by its total, and write the total down



Write the 18 down. At the end, multiply it back in and nothing is lost.

Condition 3 broken: divide a list by its total, and write the total down



Throw the 18 away and the shares are still right, but the totals, and so Z, are gone. Stated:

Proposition 2: rescaling changes no share, only the totals

Proposition 2 (normalisation invariance). Divide each message by a positive number as it is formed. Every share $\tilde{b}_i(x_i)/\sum_{x_i}\tilde{b}_i(x_i)$ is unchanged. Each total $\sum_{x_i}\tilde{b}_i(x_i)$ is Z divided by the product of the numbers used on the way to X_i.

A constant factor on one list multiplies every entry of every later list by the same amount, and a share is a ratio of entries, so it never sees that factor.

Proposition 2: rescaling changes no share, only the totals

Proposition 2 (normalisation invariance). Divide each message by a positive number as it is formed. Every share $\tilde{b}_i(x_i) / \sum_{x_i} \tilde{b}_i(x_i)$ is unchanged. Each total $\sum_{x_i} \tilde{b}_i(x_i)$ is Z divided by the product of the numbers used on the way to X_i.

That is why Theorem 1 said “without rescaling”: with it, Z comes back only if the numbers were written down. Check it on the running model.

Check it: divide the three arriving lists, then multiply the totals back

divide : $\frac{(12,6)}{18}$, $\frac{(1,4)}{5}$, $\frac{(10,20)}{30}$

written down: $18, 5, 30$

Each list divided by its own total, and the three totals written down.

Check it: divide the three arriving lists, then multiply the totals back

divide : $\frac{(12,6)}{18}, \quad \frac{(1,4)}{5}, \quad \frac{(10,20)}{30}$

written down: $18, 5, 30$

belief : $\frac{(12,6)(1,4)(10,20)}{18 \cdot 5 \cdot 30} = \frac{(120,480)}{2700}$

2700 times smaller

The belief at X_2 is the product of the three divided lists.

Check it: divide the three arriving lists, then multiply the totals back

divide :	$\frac{(12,6)}{18}, \quad \frac{(1,4)}{5}, \quad \frac{(10,20)}{30}$	written down: $18, 5, 30$
belief :	$\frac{(12,6)(1,4)(10,20)}{18 \cdot 5 \cdot 30} = \frac{(120,480)}{2700}$	2700 times smaller
share :	$\frac{(120,480)}{600} = (1/5,\ 4/5)$	unchanged

The share is a ratio of the two entries: the 2700 cancels. Proposition 2's first half.

Check it: divide the three arriving lists, then multiply the totals back

divide :	$\frac{(12,6)}{18}, \quad \frac{(1,4)}{5}, \quad \frac{(10,20)}{30}$	written down: $18, 5, 30$
belief :	$\frac{(12,6)(1,4)(10,20)}{18 \cdot 5 \cdot 30} = \frac{(120,480)}{2700}$	2700 times smaller
share :	$\frac{(120,480)}{600} = (1/5,\ 4/5)$	unchanged
total :	$\frac{600}{2700} = \frac{2}{9}, \quad Z = \frac{2}{9} \times 2700 = 600$	multiply back

The total is $Z/2700$; multiplying the written-down numbers back gives Z. Proposition 2's second half. Condition 2 we keep: every message by the rules. What does that cost?

Condition 2 kept: the cost follows the largest table, not the number of variables

Proposition 3 (cost). One message out of f costs order $\prod_{j \in \mathcal{N}(f)} |\mathcal{X}_j|$ operations, the size of f's table; the number of messages is $2|\mathcal{E}|$. Listing every case costs $\prod_{i \in \mathcal{I}} |\mathcal{X}_i|$.

Example: $m_{f_{245} \to 2}$ reads twelve entries; $m_{f_{12} \to 2}$ four. Both were computed once.

Condition 2 kept: the cost follows the largest table, not the number of variables

Proposition 3 (cost). One message out of f costs order $\prod_{j \in \mathcal{N}(f)} |\mathcal{X}_j|$ operations, the size of f's table; the number of messages is $2|\mathcal{E}|$. Listing every case costs $\prod_{i \in \mathcal{J}} |\mathcal{X}_i|$.

Exponential in the largest table, linear in the edges. The speech lattice costs frames times symbols, not 10^{338}. Count it on the running model.

Counted on the running model

	messages	multiplications	additions	operations
one root, X_2	11	36	14	50

The inward sweep; the six copies and the four leaf tables cost nothing.

Counted on the running model

	messages	multiplications	additions	operations
one root, X_2	11	36	14	50
all five beliefs	22	100	37	137

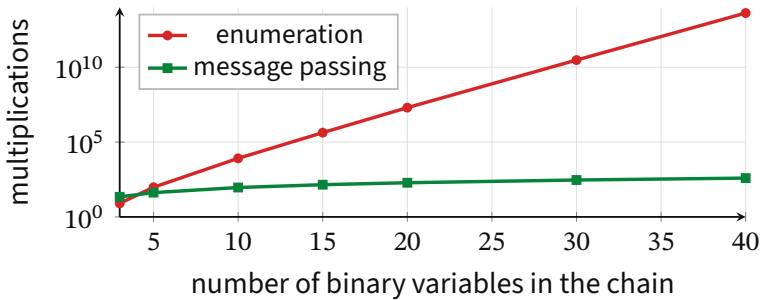
$137/50 \approx 2.7$, not 2: a local function of degree d sends one message in the first row and d in the second.

Counted on the running model

	messages	multiplications	additions	operations
one root, X_2	11	36	14	50
all five beliefs	22	100	37	137
listing every case	—	144	240	384

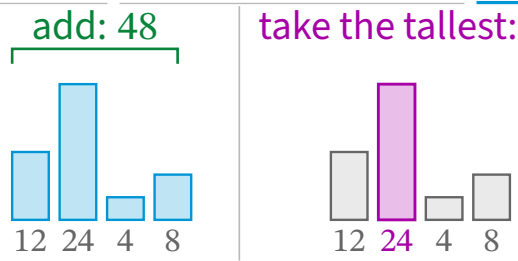
On five variables the gap is small. Draw it for forty.

Listing every case leaves the chart before twenty variables



Unit 1's picture with Proposition 3's count. The proof used + and × only through Lemma 1; a speech decoder needs $\max$ in place of +.

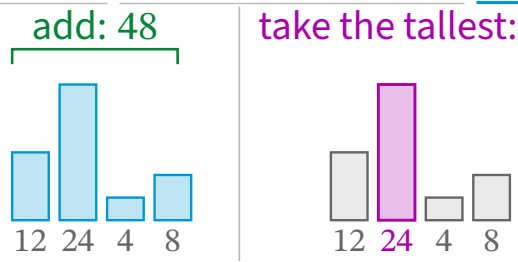
Two different questions about the same four products



$$g_2(0) = \sum_{x_1, x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) = 12 + 24 + 4 + 8 = 48$$

Unit 1's four products for $x_2 = 0$: $12, 24, 4, 8$. Adding asks how much weight the group carries.

Two different questions about the same four products



$$g_2(0) = \sum_{x_1,x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) = 12 + 24 + 4 + 8 = 48$$

$$M_2(0) = \max_{x_1,x_3} f_{12}(x_1, 0)\, f_{23}(0, x_3) = \max(12, 24, 4, 8) = 24$$

Taking the largest asks which single case is the heaviest. New question, new vocabulary.

The second question brings its own vocabulary

Definition 7 (max-marginal and MAP assignment). $M_i(x_i)$ is the largest weight of any complete assignment with $X_i = x_i$. A **MAP assignment**, maximum a posteriori, is a complete assignment of greatest weight, and $g^* = \max_x g(x)$ is that weight.

Example on the three-machine chain: $M_2(0) = 24$, the tallest bar on the previous slide; $M_2(1) = \max(1, 4, 5, 20) = 20$; so $g^* = 24$ at $(x_1, x_2, x_3) = (1, 0, 0)$.

The second question brings its own vocabulary

Definition 7 (max-marginal and MAP assignment). $M_i(x_i)$ is the largest weight of any complete assignment with $X_i = x_i$. A **MAP assignment**, maximum a posteriori, is a complete assignment of greatest weight, and $g^* = \max_x g(x)$ is that weight.

Can the sweeps compute M_i? Lemma 1 with $\max$ in place of $+$ was promised in Unit 1.

One branch, both aggregations, side by side

$$\sum_{x_1} f_{12}(x_1, 0) = 4 + 8 = 12$$

The message out of f_{12} for $x_2 = 0$, as in Unit 1: add the column.

One branch, both aggregations, side by side

$$\sum_{x_1} f_{12}(x_1, 0) = 4 + 8 = 12$$

$$\max_{x_1} f_{12}(x_1, 0) = \max(4, 8) = 8, \text{ won by } x_1 = 1$$

The same column; keep the largest instead, and remember who won.

One branch, both aggregations, side by side

$$\sum_{x_1} f_{12}(x_1, 0) = 4 + 8 = 12$$

$$\max_{x_1} f_{12}(x_1, 0) = \max(4, 8) = 8, \text{ won by } x_1 = 1$$

Aggregation: the rule that turns a column of numbers into one. Change it in both rules, and every message remembers its winner. Run the sweep.

Run the inward sweep with maxima

$m^*_{f_{12}\to 2}$ | 8 | 5 |

$m^*_{f_{23}\to 2}$ | 1 | 4 |

$m^*_{f_{245}\to 2}$ | 4 | 9 |

The largest entry per column instead of the sum: $\max(4, 8)$, $\max(1, 5)$; then $(1, 4)$; then the largest of each half of f_{245}.

Multiply entry by entry. What is the largest weight in the model?

Run the inward sweep with maxima

$m^*_{f_{12}\to 2}$ | 8 | 5 |

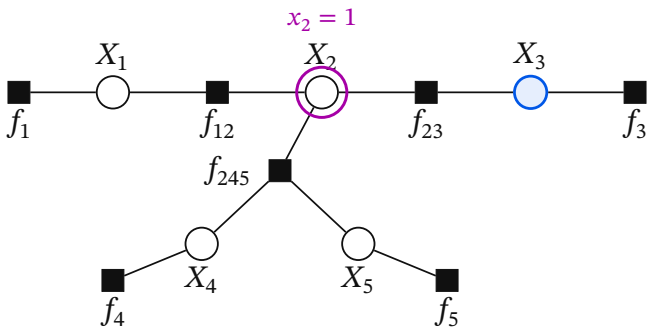
$m^*_{f_{23}\to 2}$ | 1 | 4 |

$m^*_{f_{245}\to 2}$ | 4 | 9 |

The largest entry per column instead of the sum: $\max(4, 8)$, $\max(1, 5)$; then $(1, 4)$; then the largest of each half of f_{245}.

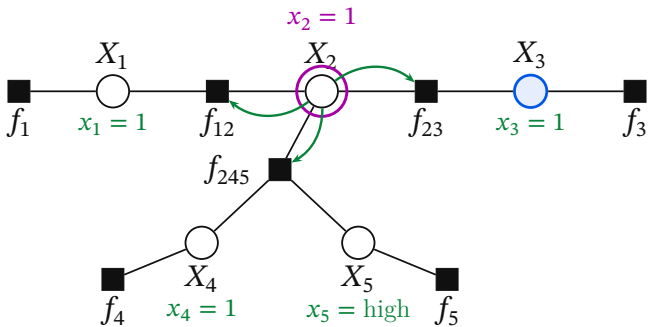
Multiply entry by entry. What is the largest weight in the model?

The root chooses, and the stored choices unroll outward



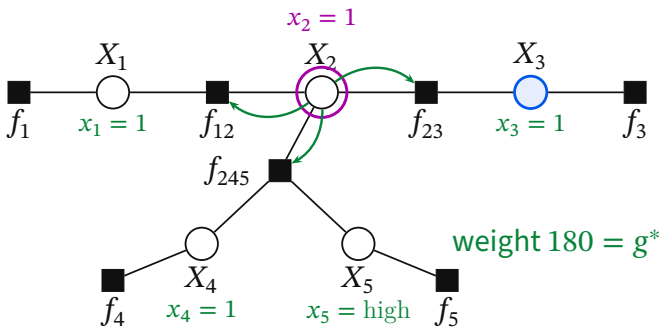
X_2 picks the larger entry of $(32, 180)$: $x_2 = 1$.

The root chooses, and the stored choices unroll outward



Each local function stored, for $x_2 = 1$, which value of its other variables won: f_{12} stored $x_1 = 1$, f_{23} stored $x_3 = 1$, f_{245} stored $(x_4, x_5) = (1, \text{high})$.

The root chooses, and the stored choices unroll outward



Read them out: $(1, 1, 1, 1, \text{high})$, weight 180. Check it against brute force.

Check the recovered assignment against brute force

$$(x_1, x_2, x_3, x_4, x_5) = (1, 1, 1, 1, \text{high})$$

What weight does it carry, and does listing all 48 agree?

Check the recovered assignment against brute force

$$(x_1, x_2, x_3, x_4, x_5) = (1, 1, 1, 1, \text{high})$$

What weight does it carry, and does listing all 48 agree?

Each local function stored one choice per value of the receiver

stored choice	for $x_2 = 0$	for $x_2 = 1$
f_{12} chooses x_1	1	1
f_{23} chooses x_3	1	1
f_{245} chooses (x_4, x_5)	$(0, \text{low})$	$(1, \text{high})$

Where the entries come from: when m^* took the largest of a column, the table wrote down who won. f_{12}: $\max(4, 8)$ and $\max(1, 5)$, both won by $x_1 = 1$.

Each local function stored one choice per value of the receiver

stored choice	for $x_2 = 0$	for $x_2 = 1$
f_{12} chooses x_1	1	1
f_{23} chooses x_3	1	1
f_{245} chooses (x_4, x_5)	$(0, \text{low})$	$(1, \text{high})$

f_{23}: f_3 zeroes $x_3 = 0$, so $x_3 = 1$ wins twice. f_{245}: 4 sits at $(0, \text{low})$ in one half, 9 at $(1, \text{high})$ in the other.

Each local function stored one choice per value of the receiver

stored choice	for $x_2 = 0$	for $x_2 = 1$
f_{12} chooses x_1	1	1
f_{23} chooses x_3	1	1
f_{245} chooses (x_4, x_5)	$(0, \text{low})$	$(1, \text{high})$

Only the column of the value the root chose is ever read; a Viterbi decoder stores exactly this table, one column per step. Why must the root choose before any branch is read?

Why must the root choose first?

Column $x_2 = 0$ of the table holds the best completion *if* $x_2 = 0$; column $x_2 = 1$ holds the best completion *if* $x_2 = 1$.

Why can the branches not read their choices before X_2 has chosen?

Why must the root choose first?

Column $x_2 = 0$ of the table holds the best completion *if* $x_2 = 0$; column $x_2 = 1$ holds the best completion *if* $x_2 = 1$.

Why can the branches not read their choices before X_2 has chosen?

Taking each variable's own best value happens to work here

That is a coincidence, not a theorem.

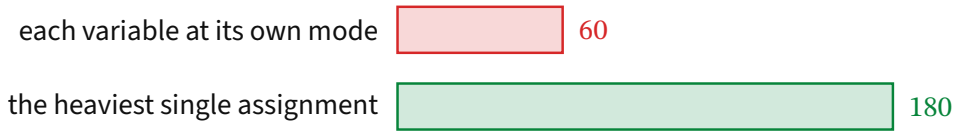
On this model every M_i peaks at the value the backtrack chose. Change one entry: $f_{245}(1, 0, \text{low})$ from 1 to 8. Nothing else moves.

Taking each variable's own best value happens to work here

That is a coincidence, not a theorem.

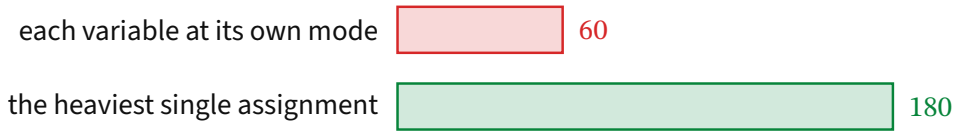
Now assemble each variable’s own most probable value and see.

First failure: the per-variable winners disagree



With the changed entry, $Z = 768$ and X_4's belief is $(396, 372)$. Assemble each variable's most probable value. What weight does it carry?

First failure: the per-variable winners disagree



With the changed entry, $Z = 768$ and X_4's belief is $(396, 372)$. Assemble each variable's most probable value. What weight does it carry?

Second failure: two assignments tie

M_4	180	180	
M_5	180	80	180

Set that entry to 9 instead. If each variable chooses alone, what can go wrong?

Second failure: two assignments tie

M_4	180	180	
M_5	180	80	180

Set that entry to 9 instead. If each variable chooses alone, what can go wrong?

Theorem 2 needs only that products distribute over maxima

Theorem 2 (max-product on a tree). Since $a \max(u, v) = \max(au, av)$ for $a \geq 0$, the proof of Theorem 1 goes through with $\max$ for $+$: $\tilde{b}_i^* = M_i$ at every variable, and backtracking returns an assignment of weight g^*.

Lemmas 2 and 3 used Lemma 1 and nothing else about $+$; Lemma 1 holds for $\max$. Step 5 of the map makes this general.

Theorem 2 needs only that products distribute over maxima

Theorem 2 (max-product on a tree). Since $a \max(u, v) = \max(au, av)$ for $a \geq 0$, the proof of Theorem 1 goes through with $\max$ for $+$: $\tilde{b}_i^* = M_i$ at every variable, and backtracking returns an assignment of weight g^*.

Test the weight achieved, never the identity of the assignment: ties are legitimate. Both questions, side by side on the model:

Both questions, answered on the same model

	sum-product	max-product
X_1	$(120, 480)$	$(36, 180)$
X_2	$(120, 480)$	$(32, 180)$
X_3	$(0, 600)$	$(0, 180)$
X_4	$(228, 372)$	$(60, 180)$
X_5	$(108, 180, 312)$	$(32, 80, 180)$
total or maximum	$600 = Z$	$180 = g^*$

Left: every row adds to Z (Theorem 1). Right: every row peaks at g^* (Theorem 2).

Both questions, answered on the same model

	sum-product	max-product
X_1	$(120, 480)$	$(36, 180)$
X_2	$(120, 480)$	$(32, 180)$
X_3	$(0, 600)$	$(0, 180)$
X_4	$(228, 372)$	$(60, 180)$
X_5	$(108, 180, 312)$	$(32, 80, 180)$
total or maximum	$600 = Z$	$180 = g^*$

On a chain the left column is the forward–backward algorithm and the right is Viterbi. Four ways to get them wrong:

Three of these four errors still normalise

correct 12 24 wrong axis 5 52

Wrong table axis	summed over the receiver (Unit 1)
Destination echoed	one table counted twice (Unit 2)
Normalised, then Z	the factors were discarded (Prop. 2)
Divided before checking	zero weight means impossible (Unit 2)

Checkpoint: Unit 3, step 4

Claim	On a tree the beliefs are the group totals, each row adding to Z (Theorem 1); with $\max$, each row peaks at g^* (Theorem 2).
Condition	No cycle, every message computed, factors kept.
Failure	On a cycle the same updates settle on a wrong answer.
Next	The proof used $+$ and $\times$ only through Lemma 1. Step 5: what else satisfies Lemma 1?

Break, 10 minutes

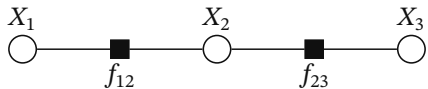
Back for steps 5 and 6: the same theorem with $\max$ and with a repaired cycle, then the lattice of the opening in today's words.

Where we are

1. Why: a loss that is a sum with 10^{338} terms
2. One step by hand: pull the shared factor out, close a box
3. The whole method on a tree: every box becomes a list on an edge; two sweeps
4. Why it is exact: the theorem, its conditions, how it breaks
5. **The same theorem with $\max$; with a cycle; with continuous values**
6. Where it is used

Step 5 asks one question of the proof just given: which facts about $+$ and $\times$ did it actually use?

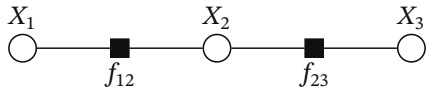
Which facts about $+$ and $\times$ did the proof use?



Lemma 1: $4 \cdot 3 + 4 \cdot 1 = 4 \cdot (3 + 1)$ $a \times (b + c) = a \times b + a \times c$

Every cut in the proof of Lemma 3 was Lemma 1: the shared factor pulled out of a sum.

Which facts about $+$ and $\times$ did the proof use?

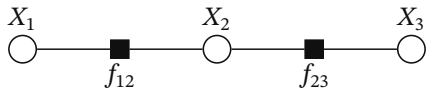


Lemma 1: $4 \cdot 3 + 4 \cdot 1 = 4 \cdot (3 + 1)$ $a \times (b + c) = a \times b + a \times c$

reorder: $\sum_{x_1} \sum_{x_3} = \sum_{x_3} \sum_{x_1}$ $a + b = b + a, (a + b) + c = a + (b + c)$

Every proof also moved sums past each other and dropped the brackets between them.

Which facts about $+$ and $\times$ did the proof use?



Lemma 1: $4 \cdot 3 + 4 \cdot 1 = 4 \cdot (3 + 1)$ $a \times (b + c) = a \times b + a \times c$

reorder: $\sum_{x_1} \sum_{x_3} = \sum_{x_3} \sum_{x_1}$ $a + b = b + a, (a + b) + c = a + (b + c)$

close a box: $(f_{12} f_{23}) f_3 = f_{12} (f_{23} f_3)$ the same two facts for $\times$

Three facts, and nothing else: no division, no subtraction. Does $\max$ have the same three?

Definition 8: $\max$ passes the same three tests, so it gets the same name

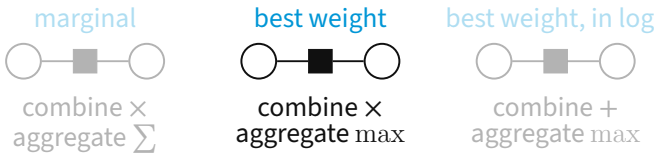
Definition 8 (commutative semiring). Two operations $(\oplus, \otimes)$, each associative and commutative, with $\otimes$ distributing over $\oplus$: $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$.



$(+, \times)$: the group totals, $g_2 = (120, 480)$.

Definition 8: $\max$ passes the same three tests, so it gets the same name

Definition 8 (commutative semiring). Two operations $(\oplus, \otimes)$, each associative and commutative, with $\otimes$ distributing over $\oplus$: $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$.



$(\max, \times)$: $4 \cdot \max(3, 1) = \max(12, 4)$; the largest weights, $M_2 = (32, 180)$.

Definition 8: $\max$ passes the same three tests, so it gets the same name

Definition 8 (commutative semiring). Two operations $(\oplus, \otimes)$, each associative and commutative, with $\otimes$ distributing over $\oplus$: $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$.



$(\max, +)$ on logarithms, as $\log(4 \cdot 5) = \log 4 + \log 5$. Now restate the theorem:

Theorem 3 says the algorithm never needed sums

Theorem 3 (sum-product over a commutative semiring). Let $(\oplus, \otimes)$ be a commutative semiring, written for $+, \times$ in Definition 4 and in the two rules. On a tree every belief is the $\oplus$-total $\tilde{b}_i(x_i) = \bigoplus_{x_j,\, j \neq i} \bigotimes_{f \in \mathcal{F}} f(x_{\mathcal{N}(f)})$.

Proof: homework, by rewriting Lemmas 2 and 3 with $\oplus, \otimes$.

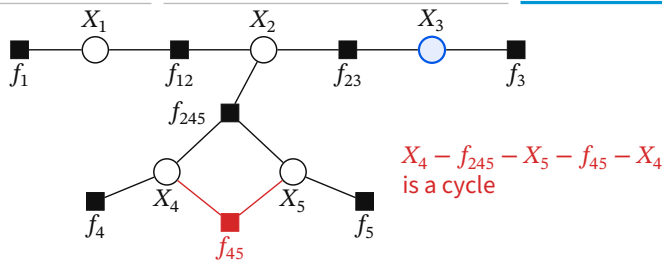
Example, $(\max, +)$ on the model: every belief peaks at $\ln 180 = 5.192957$, at $(1, 1, 1, 1, \text{high})$ as before.

Theorem 3 says the algorithm never needed sums

Theorem 3 (sum-product over a commutative semiring). Let $(\oplus, \otimes)$ be a commutative semiring, written for $+, \times$ in Definition 4 and in the two rules. On a tree every belief is the $\oplus$-total $\tilde{b}_i(x_i) = \bigoplus_{x_j,\, j \neq i} \bigotimes_{f \in \mathcal{F}} f(x_{\mathcal{N}(f)})$.

That was the operations. The other condition was the tree, and step 4 watched a cycle break the rules. Can a cycle be removed instead?

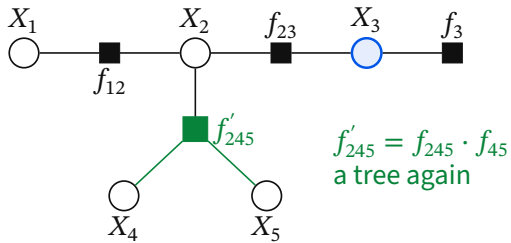
The cycle from step 4, and what listing every case says



f_{45} joins X_4 and X_5 a second time. Listing the 48 cases gives $Z = 1800$; the rules settled elsewhere.

The rules failed because f_{245} and f_{45} are two tables on one pair of variables. What if they were one table?

Multiply the two tables of the cycle into one



$$f'_{245}(x_2, x_4, x_5) = f_{245}(x_2, x_4, x_5)\, f_{45}(x_4, x_5)$$

g is unchanged: the same eight tables are multiplied, two of them first. The picture is a tree again, so Theorem 1 applies.

What does the merged table look like?

The merged table, entry by entry

$f_{45}(0, \cdot) = (5, 1, 1)$ and $f_{45}(1, \cdot) = (1, 1, 5)$: each entry of f_{245} times the entry of f_{45} under it.

x_2	x_4	x_5 = low	normal	high
0	0	$4 \cdot 5 = 20$	$2 \cdot 1 = 2$	$1 \cdot 1 = 1$
0	1	$1 \cdot 1 = 1$	$1 \cdot 1 = 1$	$1 \cdot 5 = 5$
1	0	$1 \cdot 5 = 5$	$2 \cdot 1 = 2$	$3 \cdot 1 = 3$
1	1	$1 \cdot 1 = 1$	$4 \cdot 1 = 4$	$9 \cdot 5 = 45$

Still twelve entries, still one table of three variables: nothing grew. Run the two sweeps on the repaired tree. What must come out?

Check that the repair is exact

The repaired tree: $f_1, f_{12}, f_{23}, f_3, f'_{245}, f_4, f_5$, seven tables, eleven edges, no cycle.

Theorem 1 now applies. What total must the two sweeps return, and what shares for X_4 and X_5?

Check that the repair is exact

The repaired tree: $f_1, f_{12}, f_{23}, f_3, f'_{245}, f_4, f_5$, seven tables, eleven edges, no cycle.

Theorem 1 now applies. What total must the two sweeps return, and what shares for X_4 and X_5?

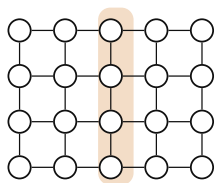
Merging until no cycle remains has a name

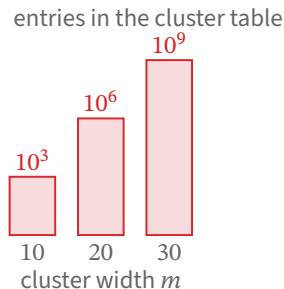
Definition 9 (cluster tree). Merge local functions that share variables until no cycle remains. The merged tables, joined through the variables they share, form a tree, and the two sweeps on it are exact by Theorem 1.

Example: $\{f_{245}, f_{45}\}$ merged into f'_{245}; the other six tables untouched. The classical construction is the junction tree.

If every cycle can be merged away, why does Week 6 exist?

The bill is the size of the merged table





f'_{245} has $2 \cdot 2 \cdot 3 = 12$ entries; a grid m wide merges into tables of 2^m. The lattice of the opening is a chain and never pays this. Week 6 approximates because merged tables grow, not because cycles are mysterious.

Proposition 4: the value sets need not be finite

Proposition 4 (continuous value sets). Let every $\mathcal{X}_i$ be an interval of the real line and every local function non-negative and integrable. With $\int$ in place of $\sum$ in Definition 4 and in the function rule, on a tree every belief is the group total $\tilde{b}_i(x_i) = \int \prod_{f \in \mathcal{F}} f(x_{\mathcal{N}(f)}) \, \mathrm{d}x_{\mathcal{I} \setminus \{i\}}$.

Proof: homework. Lemma 1 becomes $a(x) \int b(x, y) \, \mathrm{d}y = \int a(x) \, b(x, y) \, \mathrm{d}y$, and integrals reorder as sums did.

The catch: a message is now a function on an interval, not a short list. When can it still be stored?

Proposition 4: the value sets need not be finite

Proposition 4 (continuous value sets). Let every $\mathcal{X}_i$ be an interval of the real line and every local function non-negative and integrable. With $\int$ in place of $\sum$ in Definition 4 and in the function rule, on a tree every belief is the group total $\tilde{b}_i(x_i) = \int \prod_{f \in \mathcal{F}} f(x_{\mathcal{N}(f)}) \, \mathrm{d}x_{\mathcal{I} \setminus \{i\}}$.

When every table is a Gaussian, the bell curve fixed by a centre and a width: products and integrals of Gaussians are Gaussian, so each message is two numbers. The inward sweep is then the Kalman filter, the outward the smoother.

Checkpoint: Unit 3, step 5

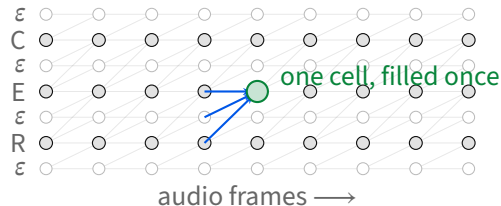
Claim	Theorem 1 survives $\max$ for $+$ (Theorem 3), a merged cycle (Definition 9) and real-valued variables (Proposition 4).
Condition	The three facts hold; merged tables stay small; messages stay storable.
Failure	A grid m wide merges into tables of 2^m entries.
Next	Step 6: the lattice of the opening, and these names, in machine learning.

Where we are

1. Why: a loss that is a sum with 10^{338} terms
2. One step by hand: pull the shared factor out, close a box
3. The whole method on a tree: every box becomes a list on an edge; two sweeps
4. Why it is exact: the theorem, its conditions, how it breaks
5. The same theorem with $\max$; with a cycle; with continuous values
6. **Where it is used**

Step 6 returns to the opening: why $451\,500$ cells give the exact sum over 10^{338} routes. Every word needed is now defined.

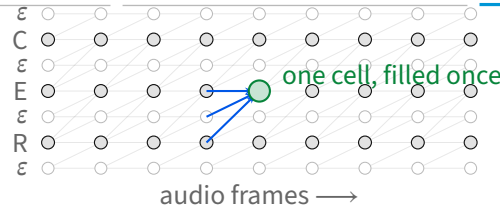
The lattice is a chain: one variable per frame, two tables per step



variables: $S_1, \ldots, S_{1500}$, $s_t \in \mathcal{S} = \{\text{the 301 rows}\}$

A route was one value for every S_t: an assignment.

The lattice is a chain: one variable per frame, two tables per step

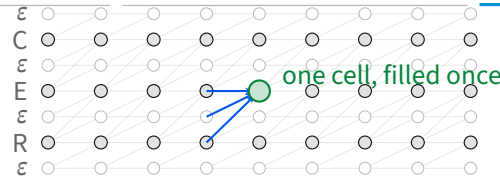


variables: $S_1, \ldots, S_{1500}$, $s_t \in \mathcal{S} = \{\text{the 301 rows}\}$

local functions: $y_t(s_t)$, the network's score, and $\mathrm{move}(s_{t-1}, s_t) \in \{0, 1\}$

Stay, advance or skip is a $0/1$ table on two neighbouring frames, like f_{12}; y_t is a table on one variable, like f_3.

The lattice is a chain: one variable per frame, two tables per step



variables: $S_1, \ldots, S_{1500}$, $s_t \in \mathcal{S} = \{\text{the 301 rows}\}$

local functions: $y_t(s_t)$, the network's score, and $\text{move}(s_{t-1}, s_t) \in \{0, 1\}$

$$p(\text{transcript}) = \sum_{s_1,\ldots,s_{1500}} \prod_t y_t(s_t)\,\text{move}(s_{t-1}, s_t) = Z$$

Multiply along a route, add over routes: the total of Definition 2, on a chain.

So what is a cell?

Say what a cell is



Part 0 called $\alpha_t(s)$ the total weight of every partial route reaching cell (t, s), filled from three earlier cells.

In this week's words, what is α_t, and what is the loss $-\log p$(transcript)?

Say what a cell is



Part 0 called $\alpha_t(s)$ the total weight of every partial route reaching cell (t, s), filled from three earlier cells.

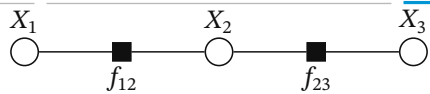
In this week's words, what is α_t, and what is the loss $-\log p$(transcript)?

The names you will meet for the objects you built

this week	the usual name
a chain, X hidden, one evidence table each	a hidden Markov model
inward message, chain	the forward quantity α_t
outward message, chain	the backward quantity β_t
a leaf's table of ones, sent unchanged	$\beta_T \equiv 1$, the backward start
rescaling, factors kept	the scaling trick
max-product, stored choices	the Viterbi algorithm
the two rules on any tree	belief propagation

Row four: nothing lies beyond a chain's last variable, so the list arriving there is all ones. What does learning need?

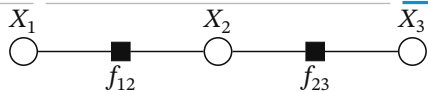
One more belief: the product of everything arriving at a local function



$$\tilde{b}_f(x_{\mathcal{N}(f)}) = f(x_{\mathcal{N}(f)}) \prod_{j \in \mathcal{N}(f)} m_{j \to f}(x_j)$$

Definition 5 multiplied the messages arriving at a variable. The same product at a local function is its belief: one entry per entry of the table.

One more belief: the product of everything arriving at a local function



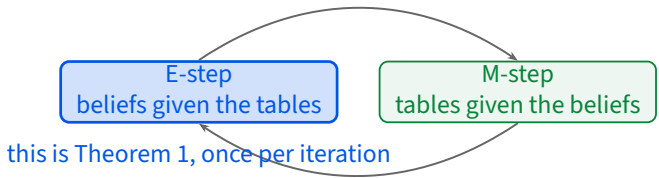
$$\tilde{b}_f(x_{\mathcal{N}(f)}) = f(x_{\mathcal{N}(f)}) \prod_{j \in \mathcal{N}(f)} m_{j \to f}(x_j)$$

$$\tilde{b}_{f_{23}}(x_2, x_3) = m_{2 \to f_{23}}(x_2)\, f_{23}(x_2, x_3)\, m_{3 \to f_{23}}(x_3)$$

$$= \begin{pmatrix} 120 \cdot 3 \cdot 0 & 120 \cdot 1 \cdot 1 \\ 120 \cdot 1 \cdot 0 & 120 \cdot 4 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 120 \\ 0 & 480 \end{pmatrix}$$

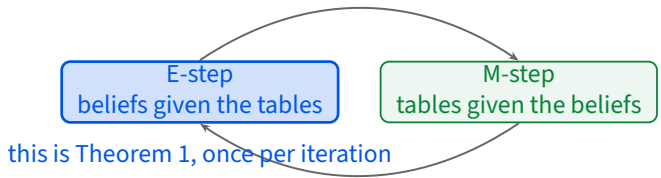
$m_{2 \to f_{23}} = (120, 120)$ and $m_{3 \to f_{23}} = f_3 = (0, 1)$ from Unit 2; the four entries add to $600 = Z$ again. Why would anyone want this table?

Where the tables come from: learning refits them from the beliefs



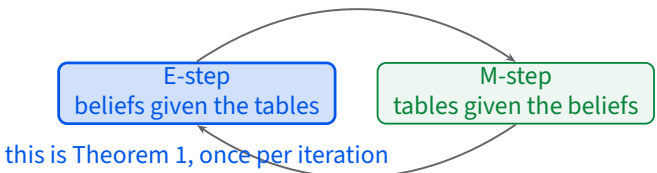
Today the seven tables were given. A trained model fits them to records: $f_{23}(1,1)$ should be large if the pair $(x_2, x_3) = (1,1)$ is seen often.

Where the tables come from: learning refits them from the beliefs



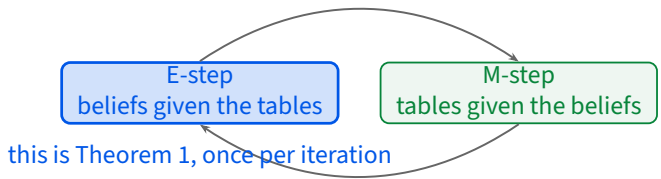
If every record showed both X_2 and X_3, fitting would be counting: $f_{23}(1,1) \propto$ the number of records with $x_2 = 1$ and $x_3 = 1$.

Where the tables come from: learning refits them from the beliefs



But X_2 is hidden: a record shows $x_3 = 1$ only. The model's own belief splits that record between the pairs: $\tilde{b}_{f_{23}}/Z = (0, 0.2;\ 0, 0.8)$, so it counts 0.2 towards $(0, 1)$ and 0.8 towards $(1, 1)$.

Where the tables come from: learning refits them from the beliefs



Guess the tables, run the two sweeps, count with the beliefs, refit, repeat: expectation maximisation. Week 5 derives the refit; today supplied its input. Which other algorithms are this?

Algorithms you know that are message passing on a tree

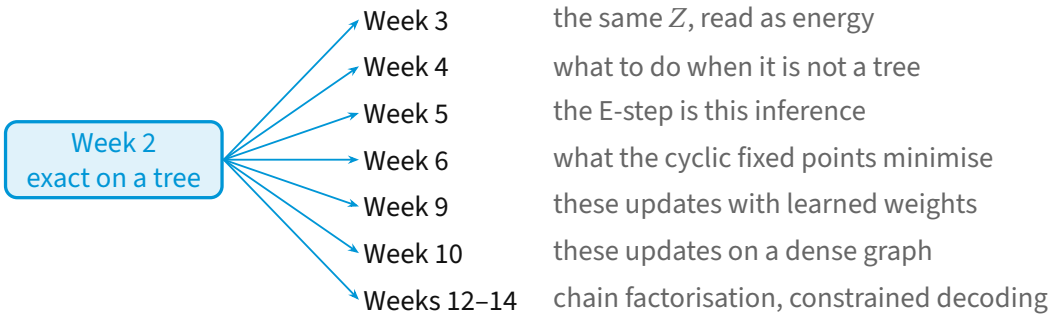
the algorithm	its graph and pair	today's name
forward–backward	a chain, $(+, \times)$	two sweeps, Theorem 1
Viterbi	a chain, $(\max, \times)$	stored choices, Theorem 2
CTC loss	the lattice, $(+, \times)$	Z from one sweep
dynamic programming	stages, $(\max, +)$	stored choices
value iteration (deterministic)	time steps, $(\max, +)$	the root chooses first
Kalman filter and smoother	real values, $(\int, \times)$	Proposition 4
expectation maximisation, E-step	any tree, $(+, \times)$	$\tilde{b}_f$, belief on a table

Dynamic programming stores the answer to a smaller problem and reuses it: the stored answer is a message. And when the graph has a cycle?

With a cycle: which part of machine learning it becomes

what is done	what it is	where
run the rules anyway	loopy belief propagation: settles on a stationary point of an approximate free energy	Week 6
merge the cycle away	the junction tree, Definition 9	Week 4
learn the update	graph neural networks: the same template, fitted weights, the destination not excluded	Week 9
value iteration, random moves	a cycle in time: the update feeds itself, and the discount $\gamma < 1$ shrinks the error each pass, so it settles correctly	reinforcement learning

What the rest of the course takes from this week



Week 6 takes the settled numbers on the cycle and asks what they minimise. Before that, this week's homework:

Homework: the proofs and checks left to you

Theorem 3, Proposition 4	rewrite Lemmas 2 and 3 with $\oplus$, $\otimes$, then with $\int$ for $\sum$, marking the fact each step uses
Proposition 1	prove that a message has one value whatever the schedule and whatever the root
The repaired tree	run both sweeps on f'_{245} by hand and confirm $Z = 1800$
$(\max, +)$	run the rules on the logarithms of the tables and confirm $\ln 180$
A hidden Markov model	two states, three steps, tables on the sheet: show the three beliefs share one total, then list the eight cases

This afternoon in the laboratory

Reference first	list the 48 assignments before writing any message code
One message by hand	direction, input, what it combined, what it summed away
Then break it	five implementations, each failing for one specific reason
Examined	the reference calculation, the test you designed, the defect you found, and why that test exposes it

What Week 2 established

Main result	on a tree, two sweeps give every group total exactly, and Z for free
Evidence	agreement with listing all 48 cases, and a proof that says why
Limit	on a cycle the same updates settle on a wrong answer
Next week	information and energy: what a belief actually tells you

Reading

Kschischang, Frey and Loeliger, *Factor graphs and the sum-product algorithm*, IEEE Trans. Inform. Theory 47(2):498–519, 2001, doi:10.1109/18.910572.

Pearl, *Probabilistic Reasoning in Intelligent Systems*, Morgan Kaufmann, 1988.

Koller and Friedman, *Probabilistic Graphical Models*, MIT Press, 2009, chapters 9–13.

Wainwright and Jordan, *Graphical models, exponential families, and variational inference*, Found. Trends Mach. Learn. 1(1–2):1–305, 2008, doi:10.1561/2200000001.

Graves, Fernandez, Gomez and Schmidhuber, *Connectionist temporal classification*, ICML 2006, doi:10.1145/1143844.1143891.

Exit question

A program is asked for the probability that Machine 2 alerts. It returns the single most likely configuration of all five variables.

Is this a numerical error, and what would you change?

Exit question

A program is asked for the probability that Machine 2 alerts. It returns the single most likely configuration of all five variables.

Is this a numerical error, and what would you change?